CHAPTER 5

THEORY OF COMPUTATION

Syllabus: Regular languages and finite automata, context-free languages and pushdown automata, recursively enumerable sets and Turing machines, undecidability.

5.1 INTRODUCTION

Theory of computation is a branch of computer science and mathematics that studies whether a problem can be solved using an algorithm on a model of computation. This branch also studies that how efficiently that problem can be solved. This deals with two types of theories. First is computability theory, deals with up to which extent problem can be solved. Second is complexity theory, which deals with efficiency of solution. In this finite automata, push down automata, Turing machines, regular expressions and various grammars will be discussed.

Automata theory is the study of abstract machines (or more appropriately, abstract "mathematical" machines or systems) and the computational problems that can be solved using these machines. These abstract machines are called automata. Computability theory deals with the question of the extent to which a problem is solvable on a computer, whereas complexity theory considers not only whether a problem can be solved at all on a computer, but also how efficiently it can be solved. Two major aspects are considered in this–time complexity and space complexity.

Basic introduction and various technical terms for understanding of concept are described in the following section.

5.2 FINITE AUTOMATA

There are several things to be done in the designing and implementation of an automatic machine, but the most important question is that, what will be the behaviour of the machine? "Theory of Computation" is the subject which solves this purpose.

"Automata theory is the subject which describes the behaviour of automatic machines mathematically". In other words, we can say that "Automata Theory is the study of self-operating virtual machines to help in logical understanding of input and output process without or with intermediate stages of computation".

The model of automation is shown in Fig. 5.1.

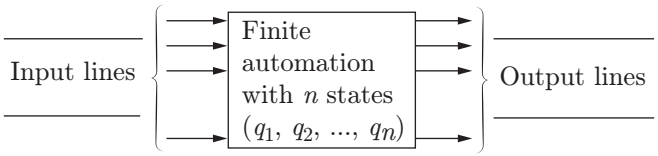


Figure 5.1 | Model of automation.

5.2.1 Finite Automaton Model Characteristics

The characteristics of finite automaton are described below:

1. **Input:** Input values from input alphabet Σ contains I_1, I_2, I_3, ... I_n at discrete time. are the input values from the input alphabet Σ at discrete time.
2. **Output:** Output O_1, O_2, O_3, ...; O_n is the output set for this computation model are different outputs of this model.
3. **States:** At any instance of time, the finite automaton will be in one of the states q_1, q_2, q_3, ..., q_n, are the set of states on which the finite automaton will be at some instance of time.
4. **States relation:** State relation describes that how to reach on next state using present input and output. At any instance of time, the next state of automaton is determined by the present input and present state.
5. **Output relation:** Either one state or number of states can be obtained as output. The next state is achieved using state relation. Either only state or both the input and state are obtained as output. When an automaton reads an input symbol, it moves to the next state, which is given by the state relation.

5.2.2 Technical Terms

Technical terms that are required in finite automaton are given as following:

1. **Alphabets:** A finite non-empty set of symbols are the alphabets of a language.

Example 5.1

$$\Sigma = \{a, b\}$$

$$\Pi = \{0, 1\}$$

Σ and Π are the sets which hold two alphabets. They are called binary alphabets for language.

$$(a + b)^* = \Sigma^* = \{\varepsilon, a, b, aa, bb, ab, ba, ...\}$$

where ε is called the identity element. Here, Σ^* contains every string possible by a, b.

$\Sigma^+ = \Sigma^* - \{\text{Null character } (\lambda)\}$, so Σ^+ will contain $\{a, b, aa, bb, ab, ...\}$.

2. **Strings:** Sequences of zero or more symbols are called string of a language. Let there be languages such as

   $L_1 = a^*$, then it will consist $\{\varepsilon, a, aa, aaa, aaaa, ...\}$ string set.
   $L_2 = ab^*$, then the string will be $\{a, ab, abb, abbb, ...\}$.
   $L_3 = 0^*1^*$, then it will consist $\{\varepsilon, 0, 1, 01, 001, 0001, 011, 0111, ...\}$ string set.
3. **Concatenation:** Let there be two strings $U = ab$ and $V = aab$. Then concatenation of these two languages is $UV = abaab$ and $VU = aabab$. Concatenations of two strings satisfy associative property but not commutative. So, $UV \neq VU$.
4. **Length of a string:** Let U, V and W be three strings. The length of a string tells the number of alphabets in that string.

   If $U = 011$, length of $|U| = 3$.
   If $V = ababa$, length of $|V| = 5$.
   If $W = abababb$, length of $|W| = 7$.
5. **Reversal of a string:** Let there be a string $W = aab$, then the reverse of W is

   $$W^r = baa$$

   If $W = W^r$, then the string is a palindrome. WW^r is called an even palindrome, and $W \times W^r$ is called an odd palindrome, where x is an alphabet. Let there be two strings U and V, then the reverse of UV is

   $$|UV|^r = V^r U^r$$
6. **Power of a string:** Let there be a string W, then

   $W^0 = \varepsilon$
   $W^1 = W$
   $W^2 = W \cdot W$
   $W^3 = W \cdot W^2$, so $W^n + 1 = W \cdot W^n = W^n \cdot W$

   Let there be a string $U = \{010\}$, then the power of U:

   $$U^0 = \varepsilon$$
   $$U^1 = \{010\}$$
   $$U^2 = \{010010\}$$
7. **Substring**: Any set of conjugative symbols is called a substring.

Example 5.2

Let $W = \{abc\}$ is a string. Substrings of W are (ε, a, b, c, ab, bc, abc), where ε is a substring of every string. If the length of a string is n, then the number of substrings possible is

$$\frac{n(n+1)}{2} + 1$$

8. **Prefix of a string:** Prefix of a string $W = \{x | xy = W\}$

   Consider a string $W = \{aabbcd\}$.
   Prefix of $W = \{\varepsilon, a, aa, aab, aabb, aabbc, aabbcd\}$
   If the length of a string is n, then the number of prefix of that string will be $(n + 1)$.

9. **Suffix of a string:** Suffix of a string $W = \{y \mid xy = W\}$

   Consider a string $W = \{aabbcd\}$.
   Suffix of $W = \{aabbcd, abbcd, bbcd, bcd, cd, d, \varepsilon\}$
   If the length of string is n, then the number of suffix of that string will be $(n + 1)$.

10. **Language:** A language is a collection of strings of finite length constructed from alphabets of symbol. Or, we can say that a language is a subset of every string made by selected alphabets.

    Let an alphabet set be $\{0, 1\}$, then Σ^* will contain all the strings made by 0 and 1. So language made by 0 and 1 will be the subset of Σ^*. Language $L \subseteq \Sigma^*$.

11. **Language union:** Let $L_1 = \{ab, ba, aab\}$ and $L_2 = \{ab, abb, bb\}$ be two languages, then $L_1 \cup L_2 = \{ab, ba, aab, abb, bb\}$.

12. **Language intersections:** Let $L_1 = \{ab, ba, aab\}$ and $L_2 = \{ab, abb, bb\}$ be two languages, then $L_1 \cap L_2 = \{ab\}$.

13. **Language complement:** Let L be a language and L' the complement of language L. So, $L' = \{\Sigma^* - L\}$

Example 5.3

Given language L as $\{aa, ab, baa\}$, then complement of language L $(L') = \{a, b\}^* - \{aa, ab, baa\}$.

14. **Language subtractions:** Let $L_1 = \{ab, ba, baa\}$ and $L_2 = \{aab, ba\}$ be two languages.

    Then $L_1 - L_2 = \{ab, baa\}$ and $L_2 - L_1 = \{aab\}$.

15. **Language XOR:** Let $L_1 = -ab, ba, baa''$ and $L_2 = -aab, ba''$ be two languages.

    $L_1 \oplus L_2 = (L_1 - L_2) \cup (L_2 - L_1) = (L_1 \cup L_2) - (L_1 \cap L_2)$
    $L_1 \oplus L_2 = \{ab, baa, aab\}$

16. **$L_1 \cdot L_2$:** Let $L_1 = \{ab, ba, aab\}$ and $L_2 = \{ba, ab\}$, then

    $L_1 \cdot L_2 = \{abba, abab, baba, baab, aabba, aabab\}$

17. **L^* and L^+:**

    $L^* = \{L^0 \cup L^1 \cup L^2 \cup L^3 \ldots\}$
    $L^+ = \{L^* - \lambda\}$

5.2.3 Grammar

Grammar is a set of rules which generates every string in a language. A grammar is described by four terms $G = (V, T, S, P)$, where

$V \rightarrow$ finite non-empty set of variables (represented by capital letters)

$S \rightarrow$ starting symbol

$T \rightarrow$ finite non-empty set of terminals (represented by small letters)

$P \rightarrow$ finite non-empty set of production rule

Example 5.4

$$S \rightarrow AB \text{ (production 1)}$$
$$A \rightarrow a \text{ (production 2)}$$
$$B \rightarrow b \text{ (production 3)}$$

where S is the starting point, A and B are variables and a, b are terminals. We have three productions here. In grammar, V, T, P should be non-empty and finite.

5.2.4 Noam Chomsky Grammar Classification

Grammar is classified into the following four types:

1. **Type 0:** This type of grammar is unrestricted, or phase structured, which generates all types of languages that can be recognized by a Turing machine. These languages are called recursive enumerable language. Unrestricted means these grammar have very less restrictions. Unrestricted grammar $(G) = u \rightarrow v$, where $u, v \in (V \cup T)^*$ and "u" must have at least one variable. V stands for variable and T for terminals.

Example 5.5

$$S \rightarrow aSb/\lambda$$
$$A \rightarrow aA/aB/Cb$$
$$Ba \rightarrow a/Da$$

2. **Type 1:** This is context-sensitive grammar (CSG) which is used to generate context-sensitive languages. CSG $(G) = u \rightarrow v$, where $u, v \in (V \cup T)^*$ and "u" must have at least one variable. There is one additional rule also, $|u| \leq |v|$, which means length of "u" is less than or equal to "v".

Example 5.6

$$S \rightarrow aSb/\lambda$$
$$Aa \rightarrow aA/aB/Cb$$
$$B \rightarrow a/Da$$

3. **Type 2:** This is context-free grammar (CFG) which is used to generate context-free languages. CFG $(G) = u \rightarrow v$, where $v \in (V \cup T)^*$ and $|u| \leq |v|$. In addition to the CSG grammar, one extra rule, that is, only single variable is allowed in "u".

Example 5.7

$$S \rightarrow aSb/\lambda$$
$$A \rightarrow aA/aB/Cb$$
$$B \rightarrow a/Da$$

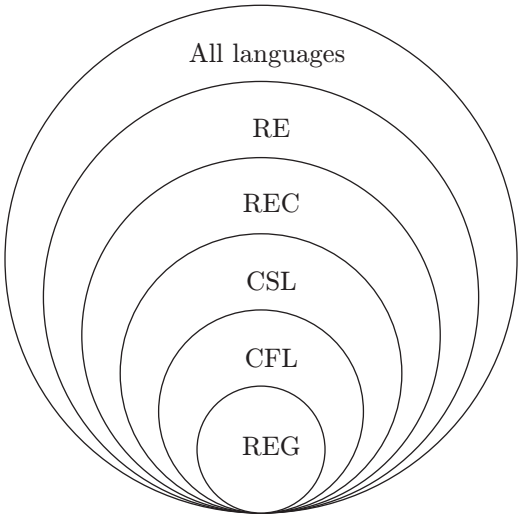
4. **Type 3:** This is regular grammar, therefore language generated by these grammars is called regular languages. Regular grammar $(G) = u \rightarrow v$, where $|u| \leq |v|$, $\{v \in (T^*, T^*V, VT^*)$ and $u \in V\}$ and "u" has only one variable.

Example 5.8

$$S \rightarrow aS/\lambda$$
$$A \rightarrow aA/aB/Cb$$
$$B \rightarrow a/Da$$

5.2.5 Chomsky Hierarchy

The hierarchy of grammar was described by Noam Chomsky in 1956, shown in Fig. 5.2.



REG—regular language; CFL—context-free language; CSL—context-sensitive language; REC—recursive language; RE—recursive enumerable.

Figure 5.2 | Chomsky hierarchy.

5.2.6 Different Grammar Types

There have been numerous grammar types proposed for different languages. Table 5.1 provides a summary of the same.

Table 5.1 | Comparison of different types of grammar

Type	Grammar	Accepted by	Restriction on Production ($x \rightarrow y$)	Example
Type 0	Unrestricted Grammar	Turing Machine	1. x must have at least one variable (V) 2. y can be any string which consist any combination of variable (V) and terminal (T)	AB → a | Ba
Type 1	Context Sensitive Grammar	Linear Bounded Automata	1. x must have at least one variable (V) 2. y can be any string which consist combination of variable (V) and terminal (T) 3. Length of y should be less than or equal to x.	Aa → aAb | aB | Cb
Type 2	Context Free Grammar	Push Down Automata	1. x can have exactly one Variable (V) 2. y can be any string which consist combination of variable (V) and terminal (T) 3. Length of y should be less than or equal to x.	A → aBa | aB | Ba
Type 3	Regular Grammar	Finite Automata	1. x can have exactly one Variable (V) 2. y can be any string which consist any combination from (VT*, T*V, T*), where V = one variable and T* = any number of terminals 3. Length of y should be less than or equal to x.	A → a | Bb | bB

5.3 FINITE AUTOMATA AND REGULAR LANGUAGE

Finite automata is classified into two categories– deterministic finite automata and non-deterministic finite automata.

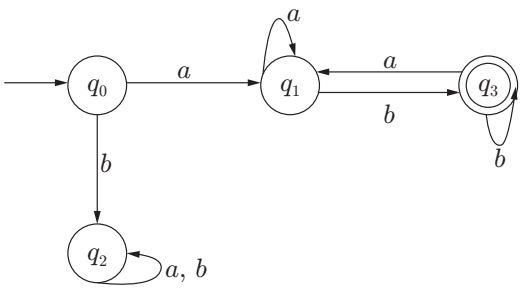
5.3.1 Deterministic Finite Automata

A deterministic finite automata (DFA) is defined by a five-tuple set $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite non-empty set of states.
2. Σ is a finite non-empty set of input called alphabets.
3. δ is a transition function which maps $Q \times \Sigma \rightarrow Q$.
4. $q_0 \in Q$, known as the initial state or starting state.
5. $F \subseteq Q$, known as a set of final states.

Example 5.9

Consider a DFA $M = (\{q_0, q_1, q_2, q_3\}\ \{a, b\}\ \{\delta\}\ \{q_0\}\ \{q_3\})$ as shown in the figure below.



The DFA can be represented in a transition table shown below, which can be determined by the transition function (δ).

δ	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_1	q_3
q_2	q_2	q_2
(q_3)	q_1	q_3

q_0 – Starting state

q_3 – Final state (accepting state)

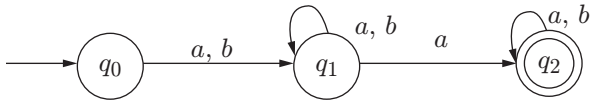
5.3.2 Non-deterministic Finite Automata

A non-deterministic finite automata (NFA) is defined by a five-tuple set $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite non-empty set of states.
2. Σ is a finite non-empty set of input called alphabets.
3. δ is a transition function which maps $Q \times (\Sigma \cup \{\lambda\}) \rightarrow 2^Q$.
4. $q_0 \in Q$, known as the initial state or starting state.
5. $F \subseteq Q$, known as a set of final states.

Example 5.10

Consider an NFA $M = (\{q_0, q_1, q_2\}\ \{a, b\}\ \{\delta\}\ \{q_0\}\ \{q_2\})$ as shown in figure below.



The NFA can be represented in a transition table shown below, which can be determined by the transition function (δ).

δ	a	b
$\rightarrow q_0$	q_1	q_1
q_1	$\{q_1, q_2\}$	q_1
(q_2)	q_2	q_2

q_0 – Starting state

q_2 – Final state (accepting state)

5.3.3 Comparison of DFA and NFA

The comparison between DFA and NFA is given in Table 5.2.

5.3.4 DFA and NFA Design

We have taken a few examples and discussed how to design a DFA and an NFA for these cases. The following are four types of states which are used to design a DFA and an NFA:

1. **Permanent accept state:** In this case, the initial and final states are the same. Due to self-loop this machine will always be on the same state, which is also final state. Hence, this is called permanent accept state.

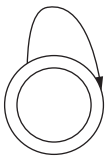
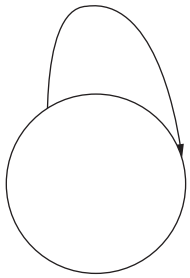


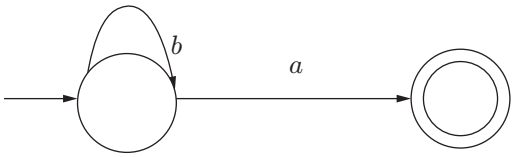
Table 5.2 | DFA vs. NFA

Title	DFA	NFA
Power	NFA and DFA powers are same	NFA and DFA powers are same
Supremacy	Some NFA are DFA, but not all	All DFA are NFA
Time complexity	Time needed to execute an input string is less than that required by NFA	Time needed to execute an input string is more than that required by DFA
Transition function	Maps $Q \times \Sigma \rightarrow Q$, number of next states at an input is exactly one	Maps $Q \times (\Sigma \cup \{\lambda\}) \rightarrow 2^Q$, the number of next states at an input is zero, one or more.
Null moves	Does not allow null moves	Allows null moves
Moves	Only one move for single input alphabet	There can be choice (more than one move can be there) for single input alphabet

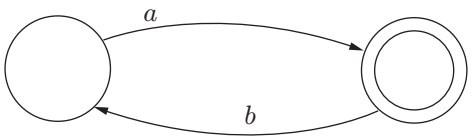
2. **Permanent reject state (trap state):** The given state is permanent reject state because there is self-loop to some non-final state.



3. **Temporary reject state:** The given state is temporary reject state because for input 'b', it has a loop on the non-final state. However, if after input 'b' there is another input 'a', then it will halt on final state.

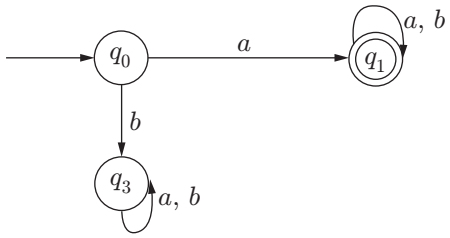


4. **Temporary accepting state:** The given state is temporary accepting state, as it reaches the final state if it has input symbol 'a', but if it gets another input symbol 'b' on that final state it will halt on non-final state.



Example 5.11

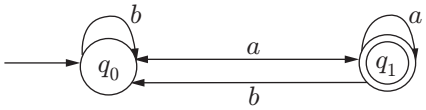
DFA which accepts string starting with "a"



Regular expression: $a(a + b)^*$

Example 5.12

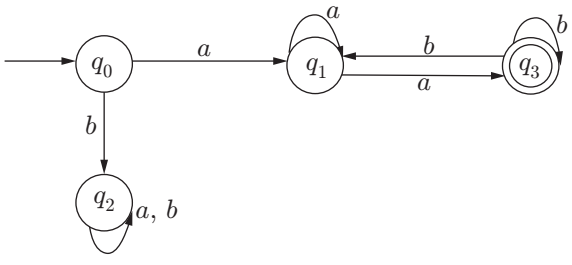
DFA which accepts string ending with "a"



Regular expression: $(a + b)^*a$

Example 5.13

DFA which accepts string starting with "a" and ending with "b"

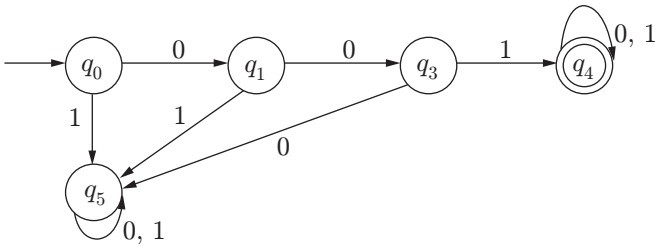


Regular expression:

1. $a(a + b)^* \cap (a + b)^*b$ (from statement)
2. $aa^*b(b + aa^*b)^*$

Example 5.14

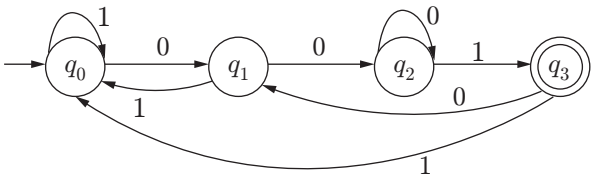
DFA which accepts string starting with "001"



Regular expression: $001(0 + 1)^*$

Example 5.15

DFA which accepts string ending with "001"



Regular expression: $(0 + 1)^*001$

5.3.5 Applications of DFA/NFA

NFA and DFA have various applications as given below:

1. Lexical analyser
2. Text editor
3. Spell checker
4. Sequential circuit design
5. Unix graph (pattern search)

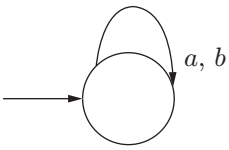
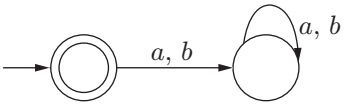
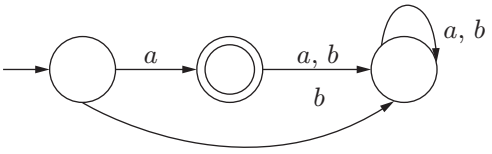
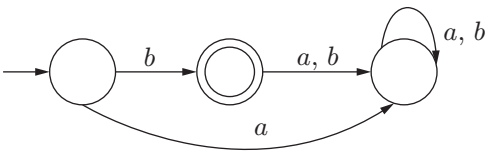
5.3.6 Regular Expression and Grammar

Finite automata accepts regular grammar. First of all, regular expressions are discussed and then how to construct regular grammar from finite automaton is described (Table 5.3).

1. **Regular expression:** A language is regular iff $\exists$ a regular expression r, that is, $L = L(r)$.
   - There can be more than one regular expression for a language L.
   - There can be more than one machine for a language L.
   - A language L can be produced by more than one grammar.

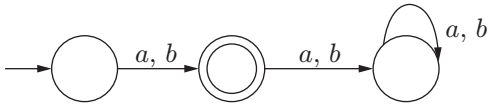
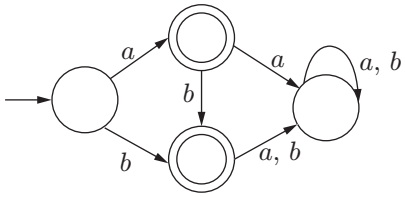
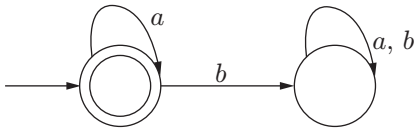
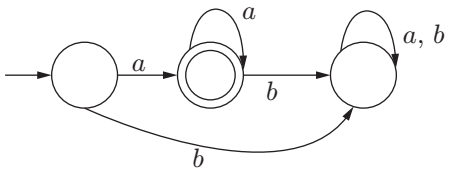
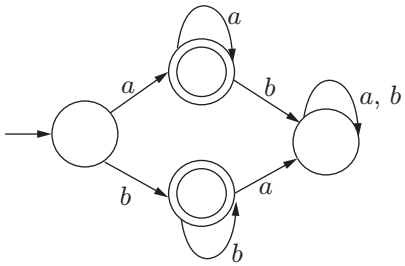
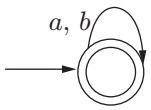
Symbols to represent regular expression are $\{a, b, \varepsilon, \Sigma, \varnothing, \lambda, +, ., *, (\ )\}$.

Table 5.3 Some standard regular languages and their regular expression, grammar and machine

Regular Expression (r)	Language (L)	Grammar (G)	Machine (M)
$\varnothing$	$\{\}$	$S \to A$	
Λ	$\{\lambda\}$	$S \to \varepsilon$	
A	$\{a\}$	$S \to a$	
B	$\{b\}$	$S \to b$	

(*Continued*)

Table 5.3 Continued

Regular Expression (r)	Language (L)	Grammar (G)	Machine (M)
$a + b$	$\{a, b\}$	$S \to a$ $S \to b$	
$a + b + ab$	$\{a, b, ab\}$	$S \to aab$ $S \to ab$ $S \to ba$	
a^*	$\{\lambda, a, aa, aaa, aaaa, \ldots\}$	$S \to aS$ $S \to \varepsilon$	
a^+	$\{a, aa, aaa, aaaa, \ldots\}$	$S \to aS$ $S \to a$	
$a^* + b^*$	$\{\lambda, a, aa, aaa, \ldots, b, bb, bbb, \ldots\}$	$S \to S_1/S_2$ $S_1 \to aS_1/\varepsilon$ $S_2 \to bS_2/\varepsilon$	
$(a + b)^*$	$\{\lambda, a, b, ab, ba, aab, aaab, baa, bab, bbb, baba, \ldots\}$	$S \to aS/bS$ $S \to a/b$	

2. **Regular grammar:** Type 3 grammar is also called regular grammar and it generates regular language. Regular grammar is divided into two types:

   Left linear grammar: $V \to VT^* + T^*$

   Right linear grammar: $V \to T^*V + T^*$

   A grammar is a regular grammar iff $\exists$ a left linear or right linear grammar for it.

5.3.7 Pumping Lemma

Pumping lemma is very useful to prove that a certain language is not a regular language. We know that the loop in finite automata (FA) makes it able to accept those strings which have length equal to and greater than its total number of states.

Let there be a string w which has a bigger length (more than its states), then we can break this string into three parts x, y (y should not be null) and z. Let FA has loop for y and $w = xyz \in L$ accepted by FA, so $w = xy^iz$ for $i = 0, 1, \ldots$ is also accepted by FA.

5.3.8 Myhill–Nerode Theorem

Myhill–Nerode theorem tells that the given language might be regular or not regular. A given language L is a regular language if and only if the set of equivalence classes of L is finite, or it satisfies following three conditions:

1. The language L divides the set of all possible strings into mutually separate classes.
2. If L is a regular language, then the number of classes created is finite.
3. If the number of classes that L creates is finite, then L is a regular language.

Problem 5.1: Consider a language L consists all strings over $\{a, b\}$ ending in b. Prove that L is a regular language.

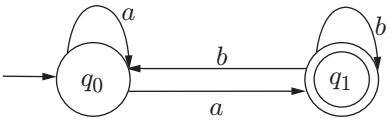
Solution: Let DFA $M = \{Q, S, \delta, q_0, F\}$ recognizes $L = \{b, bb, ab, aab, abb, \ldots\}$.

We have two classes C_0 and C_1 defined as follows:

$C_0 = \{$All strings that end in $a\}$

$C_1 = \{$All strings that end in b, i.e., accepted strings $(b, ab, abb, bbb, \ldots)\}$

Transition diagram for the DFA M is shown below.



Classes C_0 and C_1 correspond to states q_0 and q_1, respectively. Applying Myhill–Nerode theorem:

1. Two classes C_0 and C_1 are mutually exclusive.
2. L is regular and creates finite classes (here two classes).
3. The number of classes created by DFA is finite.

As all the three statements of Myhill–Nerode theorem are satisfied, hence L is regular.

5.3.9 Transducer or Finite State Machine

A finite state machine (FSM) is similar to FA except that it has the additional capability of producing an output.

$$\text{FSM} = \text{FA} + \text{Output capability}$$

There are two types of FSMs:

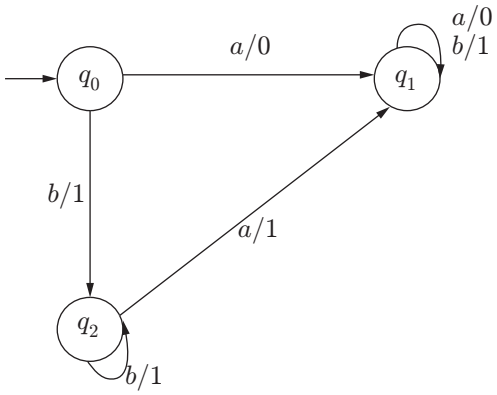
1. Mealy machines
2. Moore machines

5.3.9.1 Mealy Machines

In a Mealy machine, the output of an FSM is dependent on the present state and present input. A Mealy machine can be described by a six-tuple set $(Q, \Sigma, \Delta, \delta, \lambda, S)$, where

1. Q is a finite and non-empty set of states.
2. Σ is an input alphabet.
3. Δ is an output alphabet.
4. δ is a transition function which maps the present state and input symbol on to the next state $Q \times \Sigma \rightarrow Q$.
5. λ is the output function which maps $Q \times \Sigma \rightarrow \Delta$ ((present state + present symbol) $\rightarrow$ output).
6. $S \in Q$ is the starting state or initial state.

Problem 5.2: Consider the Mealy machine shown in the figure. Construct the transition table and find the output for input *aabba*.



Solution: Transition table for the above Mealy machine is:

Present State	Next State		Output	
	$x = a$	$x = b$	$x = a$	$x = b$
q_0	q_1	q_2	0	1
q_1	q_1	q_1	0	1
q_2	q_1	q_2	1	1

Input: *aabba*

Output: 00110

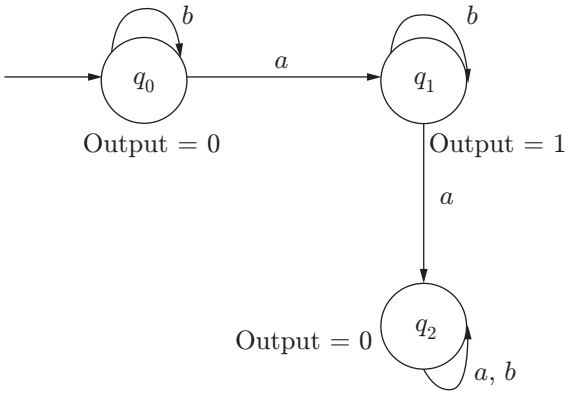
(**Note:** Output length of the Mealy machine is equal to the input length.)

5.3.9.2 Moore Machines

In Moore machine, the output of an FSM is dependent on the present state only. A Moore machine can be described by a six-tuple set $(Q, \Sigma, \Delta, \delta, \lambda, S)$, where

1. Q is a finite and non-empty set of states.
2. Σ is an input alphabet.
3. Δ is an output alphabet.
4. δ is a transition function which maps the present state and input symbol on to the next state $Q \times \Sigma \rightarrow Q$.
5. λ is the output function which maps $Q \rightarrow \Delta$ (present state $\rightarrow$ output)
6. $S \in Q$ is the starting state or initial state.

Problem 5.3: Consider the Moore machine shown in the figure. Construct the transition table and find the output for input *aabba*.



Solution: Transition table for the above Mealy machine:

Present State	Next State		Output
	$x = a$	$x = b$	
q_0	q_1	q_0	0
q_1	q_2	q_1	1
q_2	q_2	q_2	0

Input: *aabba*

Output: 010000

(**Note:** Output length of Moore machine is one greater than the input length because the first output symbol is additional without reading any symbol from the input.)

5.3.10 Conversion in Finite Automata

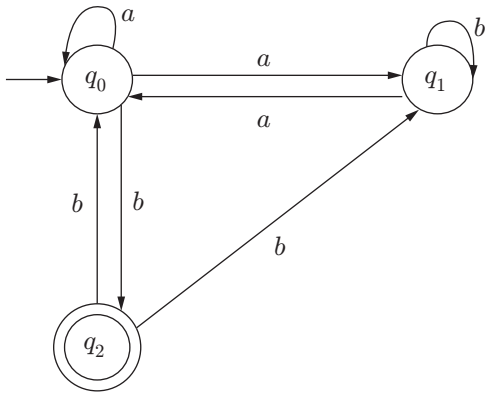
Power of NFA and DFA is the same, which means both can accept the same set of strings. Here we discuss interconversion of NFA and DFA.

5.3.10.1 NFA to DFA

To convert NFA to DFA, following points should be considered:

1. In NFA and DFA there is strictly one initial or starting state.
2. If there are n states in NFA, then equivalent DFA contains $\leq 2^n$ states.
3. DFA can simulate the behaviour of NFA by increasing the number of states.

Problem 5.4: Construct a DFA equivalent of given NFA.



Solution:

Step 1: Construct a transition table of the given NFA

δ	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	q_2
q_1	q_0	q_1
q_2	–	$\{q_0, q_1\}$

q_0 – Starting state

q_2 – Final state (accepting state)

Step 2: Construct a DFA transition table with the help of an NFA transition table. Put the first row same from the NFA transition table, then put one by one states from the right side. After the first row, put $\{q_0 q_1\}$ in the transition column and find the next state on a from q_0 and q_1 separately and write them under column a, repeat the same process for b. Now pick another state q_2 from the right side and follow the same process. This process has to be repeated until all the unique value set comes as a state under the transition table.

Now make the starting state of DFA the same state as in NFA. And make all those states final which

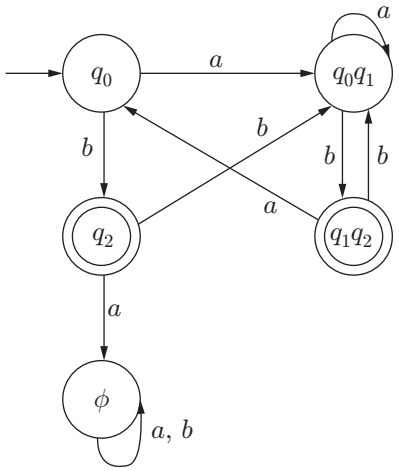
include the final state of NFA; in our example it was q_2, so q_1q_2 and q_2 states will be final states in DFA.

δ	a	b
$\rightarrow q_0$	$\{q_0q_1\}$	q_2
$\{q_0q_1\}$	$\{q_0q_1\}$	$\{q_1q_2\}$
(q_2)	ϕ	$\{q_0q_1\}$
$(\{q_1q_2\})$	q_0	$\{q_0q_1\}$
ϕ	ϕ	ϕ

→ Represent starting state

◯ Represent final or accepting state

Step 3: Draw DFA from the transition table.

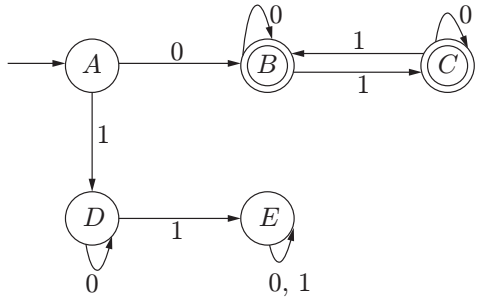


5.3.10.2 DFA to Minimum DFA

Minimization of DFA means reducing the unnecessary states. During minimization, the following steps are considered.

1. DFA with n states when converted into minimal DFA will contain N states, where $1 \leq N \leq n$.
2. We have to eliminate the same states from the given DFA to convert it into minimal DFA.
3. Two states $q_1 \equiv q_2$ iff $\forall\,(w \in \Sigma^*)$, $\{\delta(q_1, w), \delta(q_2, w)\}$ both are from the same set.
4. If a final state (accepting state) is equivalent to a non-final state, then it cannot be reduced.

Problem 5.5: Convert the given DFA into minimal DFA.



Solution:

Step 1: Make two sets of final and non-final states.

Iteration 1 = $(\{A, D, E\}\ \{B, C\})$

Step 2: Check within the set if two of them are equivalent states or not.

Let us first check in $\{A, D, E\}$, first we pick A and D.

Present State	Next State on Input	
	$X = 0$	$X = 1$
A	B	D
D	D	E
E	E	E

We can clearly see that state A is going outside on state B at input 0, which is in another set, whereas the other two states are indicating within the non-final state set. So, we will put state A in a new set.

Iteration 2 = $(\{A\}\ \{D, E\}\ \{B, C\})$

Repeat step 2 until last two iterations become same.

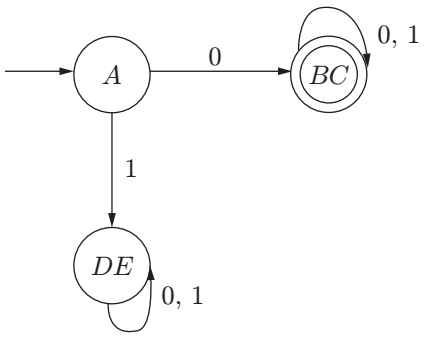
Now check for B and C.

Present State	Next State on Input	
	$X = 0$	$X = 1$
B	B	C
C	C	B

States B and C are not going outside from their own set at inputs 0 and 1, so there will be no change.

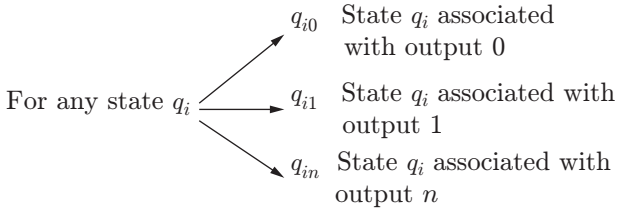
Iteration 3 = $(\{A\}\ \{D, E\}\ \{B, C\})$

We get two same iterations so we will stop this process. We will merge those states which lie in a set. There are three states in minimal DFA (A), (DE) and (BC). Now we can draw the minimal DFA.



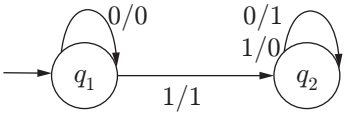
5.3.10.3 Mealy to Moore Machine

In a Mealy machine, the output depends upon both the present state and the input, and the Moore machine output depends only on the present state. While converting Mealy to Moore we develop a procedure so that all the states of Mealy machines, which are associated with different outputs, find them. After that, split those states into n parts if the states have n outputs.



Through this procedure all the states are associated with a single output, so all the states are considered as unique state. We can understand this concept clearly with an example.

Problem 5.6: Consider the 2's complement Mealy machine given below and convert it into Moore machine. At input "1101", the output is "0011". Input string will be read from the right side.



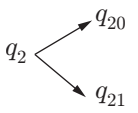
Solution: First we will make a transition table for the given Mealy machine.

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
q_1	q_1	q_2	0	1
q_2	q_2	q_2	1	0

In the transition table, we have to find those states which are associated with more than one output in the next state column.

For q_1 at $(q_1, 0)$, we have output 0.

For state q_2 at $(q_2, 0)$ output is 1 and at $(q_2, 1)$ output is 0. So there is a need to split q_2.

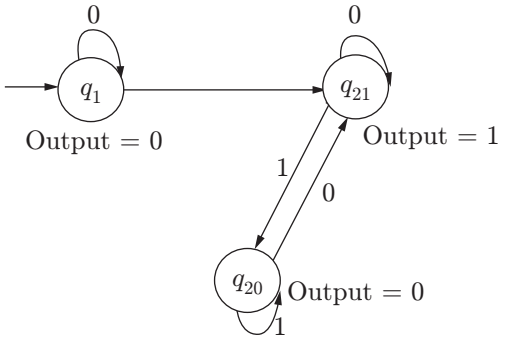


Now the Moore machine has three states $\{q_1, q_{20}, q_{21}\}$ with output $\{0, 0, 1\}$, respectively.

Now we will make a transition table of the Moore machine.

Present State	Next State		Output
	$x = 0$	$x = 1$	
$\rightarrow q_1$	q_1	q_{21}	0
q_{20}	q_{21}	q_{20}	0
q_{21}	q_{21}	q_{20}	1

Moore machine of 2's complement:



Note: m states, n outputs Mealy machine $\equiv$ Moore machine contains $\leq mn + 1$ state.

5.3.10.4 Moore to Mealy Machine

Consider the Moore machine given in Fig. 5.3 and convert it into Mealy machine.

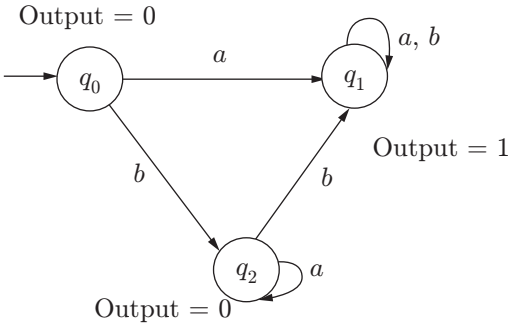


Figure 5.3 | Moore machine.

In the transition table of Moore machine (Table 5.4), the states (q_0, q_1, q_2) have outputs (0, 1, 0), respectively.

Table 5.4 | Transition table of Moore machine

Present State	Next State		Output
	$x = a$	$x = b$	
$\rightarrow q_0$	q_1	q_2	0
q_1	q_1	q_1	1
q_2	q_2	q_1	0

So, now we can make a transition table for the Mealy machine (Table 5.5).

Table 5.5 Transition table of Mealy machine

Present State	Next State		Output	
	$x = a$	$x = b$	$x = a$	$x = b$
$\rightarrow q_0$	q_1	q_2	1	0
q_1	q_1	q_1	1	1
q_2	q_2	q_1	0	1

Mealy machine equivalent to given Moore machine (Fig. 5.4):

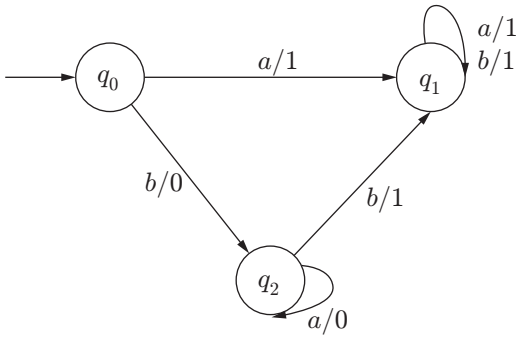


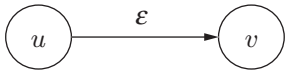
Figure 5.4 Mealy machine.

Note: m states, n outputs Moore machine $\equiv$ Mealy machine contains $\leq m$ states.

5.3.10.5 NFA with ε-Moves to NFA Without ε-Moves

There are three steps to convert NFA with ε moves to NFA without ε moves:

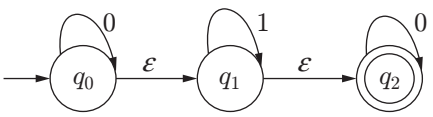
1. If there is an ε move from u to v, then every arrow starting from v is also starting from u.



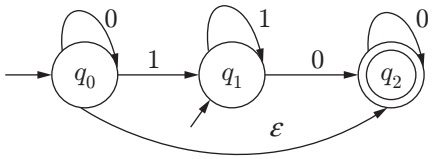
2. When ε move removes from u to v, and if u is a starting state, then make v also a starting state.
3. When ε move removes from u to v, and if v is a final state, then make u also a final state.

Problem 5.7: Convert given NFA with ε moves to NFA without ε moves.

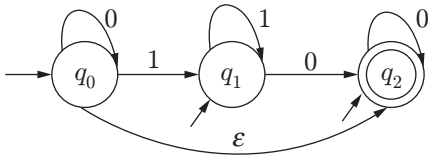
Solution:



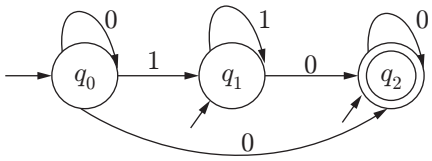
Step 1: Follow above steps to remove first ε move between q_0 and q_1.



Step 2: Follow above steps to remove ε move between q_1 and q_2.



Step 3: Follow above steps to remove ε move between q_0 and q_2.



After removing all ε moves, this is the final NFA.

Note: If NFA with ε-moves has n states then NFA without ε-moves will have exactly n states.

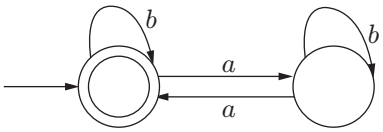
5.3.11 Properties of Regular Sets/Languages

1. Classes of two regular sets S_1 and S_2 are closed under union operation, means $S_1 \cup S_2$ is also a regular set.
2. Classes of two regular sets S_1 and S_2 are closed under concatenation operation, means S_1S_2 is also a regular set.
3. A class of regular set S_1 is closed under Kleene closure operation, means S_1^* is also a regular set.
4. If S is a regular set on some alphabet Σ, then complement of S is denoted by $\overline{S}$ is also a regular set. Steps to make complement of an NFA are as follows:
   - Change all final states to non-final states of NFA for Σ.
   - Change all non-final states to final states of NFA for Σ.
5. Classes of two regular sets S_1 and S_2 are closed under intersection operation, means $S_1 \cap S_2$ is also a regular set.
6. If S is a regular set on some alphabet Σ, then transpose of S is denoted by S^R is also a regular set.

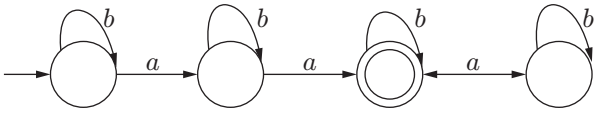
5.3.12 Some Important DFA

Some examples of important DFAs are given below:

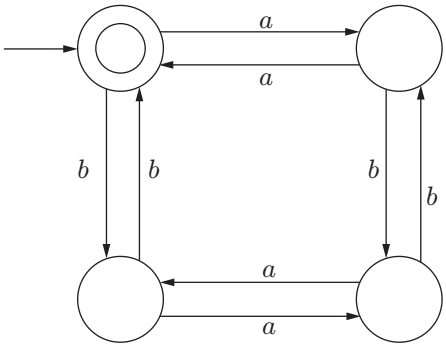
1. Containing even number of a's. This can be written as $L = \{w \mid na(w) \bmod 2 = 0\}$:



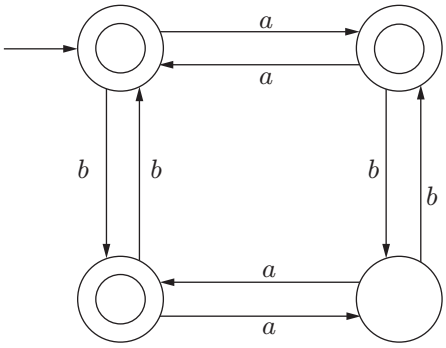
2. DFA having exactly two a's:



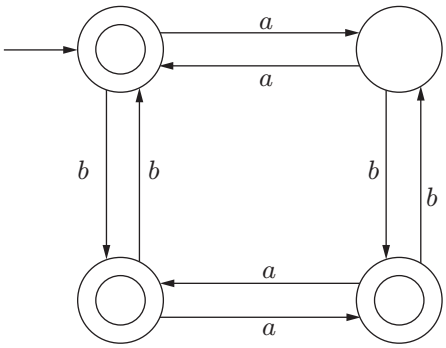
3. $L = \{w \mid na(w) \bmod 2 = 0 \text{ and } nb(w) \bmod 2 = 0\}$:



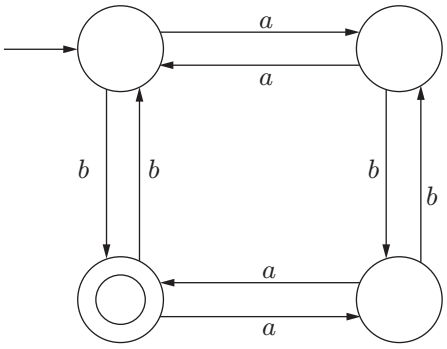
4. $L = \{w \mid na(w) \bmod 2 = 0 \text{ or } nb(w) \bmod 2 = 0\}$:



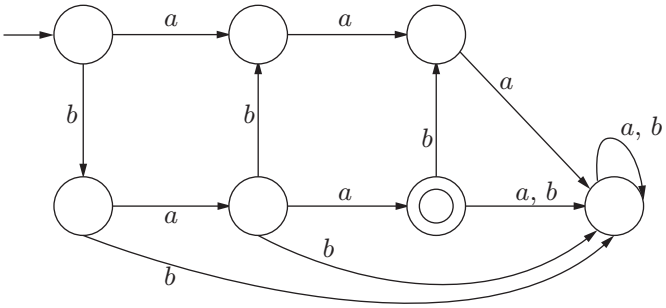
5. $L = \{w \mid na(w) \bmod 2 = 0 \text{ or } nb(w) \bmod 2 = 1\}$:



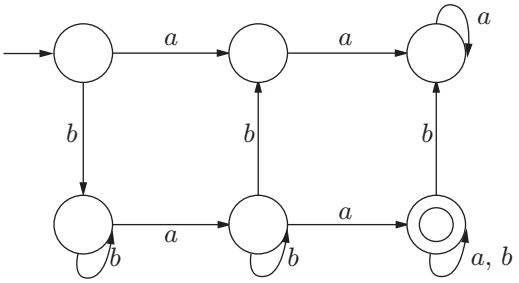
6. $L = \{w \mid na(w) \bmod 2 = 0 \text{ and } nb(w) \bmod 2 = 1\}$:



7. DFA which accepts exactly two a's and one b:



8. DFA which accepts at least two a's and at least one "b":



5.4 PUSHDOWN AUTOMATA

Pushdown automata (PDA) are devices that recognize context-free languages. PDA has finite memory in terms of a stack. A pushdown automaton is a non-deterministic device. The deterministic version of automata accepts only a subset of CFL known as deterministic context-free language (DCFL).

5.4.1 Model of PDA

PDA can be defined as a seven-tuple set $\{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$, where

1. Q is a finite and non-empty set of states.
2. Σ is a finite and non-empty set of input symbol.

3. Γ is a finite non-empty set of stack alphabets.
4. δ is the transition function which maps $Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow Q \times \Gamma^*$
5. q_0 is the initial state (start state).
6. $Z_0 \in \Gamma$ is the starting (top most) stack symbol.
7. F is the set of final states and $F \subseteq Q$.

Figure 5.5 shows a model of PDA.

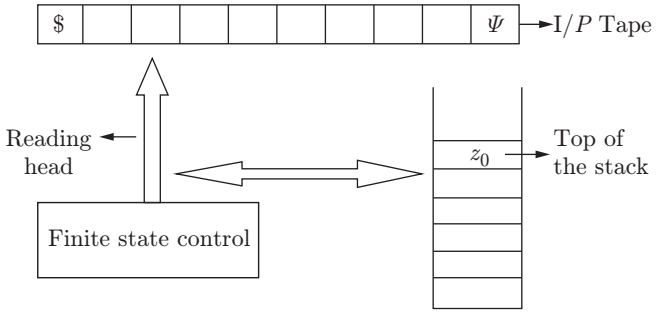


Figure 5.5 Model of PDA.

5.4.2 Transition Function

Three types of operations are performed under the transition function (δ) of PDA.

1. **Push element into stack:** Let the current stack symbol be z_0, current state be q_0 and current input symbol be "a". So after reading "a", z_2 is to be pushed into stack and next state may or may not be changed according to the transition function written as

   δ(current state, current input symbol, top of stack symbol) $\rightarrow$ (next state, new top of stack)

   $$\delta(q_0, a, z_0) = (q_1, az_0)$$

2. **Pop element from stack:** If "a" is input symbol on state q_0 with stack top z_1 and z_1 is popped up after processing "a" and next state is q_1. Then transition function δ is written as

   δ(current state, current input symbol, top of stack symbol) $\rightarrow$ (next state, top of stack)

   $$\delta(q_0, a, z_1) = (q_1, \lambda)$$

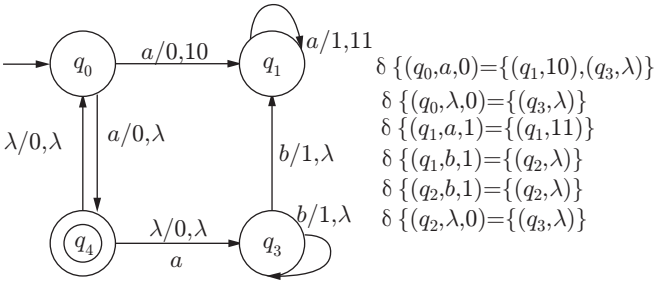
3. **No change in state element:** If on an input symbol "a" at state q_0 on stack top z_1 nothing is pushed or popped, then transition function can be written as

   $$\delta(q_0, a, z_1) = (q_1, z_1)$$

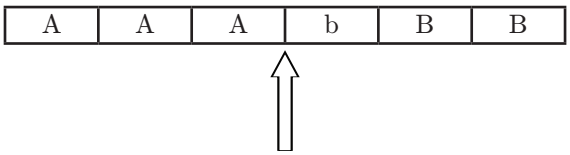
5.4.3 Non-deterministic PDA

Non-deterministic pushdown automata (NPDA) have more than one choice for a single input such as NFA. Transition function of NPDA is $Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow Q \times \Gamma^*$. An NPDA accepts an input string if a sequence leads to some final state of PDA or cause PDA to empty its stack. A non-deterministic automaton is equivalent to a CFG and more powerful than a deterministic PDA.

Example 5.16



This NPDA will recognize the string "*aaabbb*" by the following moves:



Input head start from left and starting stack symbol is 0.

1. $(q_0, a, 0) \vdash. (q_1, 10)$
2. $(q_1, a, 10) \vdash. (q_1, 110)$
3. $(q_1, a, 110) \vdash. (q_1, 1110)$
4. $(q_1, b, 1110) \vdash. (q_2, 110)$
5. $(q_2, b, 110) \vdash. (q_2, 10)$
6. $(q_2, b, 10) \vdash. (q_2, 0)$
7. $(q_2, \lambda, 0) \vdash. (q_3, \lambda)$

As $q_3 \in F$, the string is accepted.

5.4.4 Deterministic PDA

A deterministic pushdown automata (DPDA) is one for which every input string has a unique way through the machine. A PDA $M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0, F\}$ is deterministic if it satisfies the following two conditions:

1. $\delta(q, a, z)$ is either empty or only single move.
2. $\delta(q, \lambda, z) \neq \varnothing$ means $\delta(q, a, z) = \varnothing$ for each $a \in \Sigma$.

The language accepted by DPDA is known as deterministic context-free language (DCFL), and it is noticeable that not all CFLs are DCFLs.

Example 5.17

Language $L = \{a^n \times b^n : n \geq 0\}$, it can be accepted by the DPDA because all 'a' characters are inserted into stack till character 'x' come and then symbol 'a' is removed as many times as many symbol 'b' occur. In the last stack will be empty.

5.4.5 Parsing

Parsing is a technique to construct a parse or a derivation tree. It is to check whether there is some leftmost derivation for a given string using a certain grammar. Parse tree generated through parsing is also used by a syntax analyzer in a compiler for checking a string is according to a given grammar or not.

5.4.5.1 Top-Down Parsing

In leftmost derivation, we start with initial and replace the leftmost variable in each step and finally get the string. This approach is known as top-down parsing. In the parse, start symbol is root, terminals are leaves (input symbols) and other nodes are variables. We start from the root and replacing the intermediate nodes one by one from left to right reach the leaves. This approach is also known as recursive descent.

Here, we have constructed the leftmost derivation looking ahead one symbol in the given string. A grammar having this property is known as LL(1). In general, if a leftmost derivation is constructed by looking at the k symbol ahead in the given string, then the grammar is known as LL(k) grammar, where $k \geq 1$.

Steps of construction of top-down parser:

Step 1: Eliminate left recursion (if any) from the given grammar.

Step 2: Eliminate left factoring (if any) from the given grammar.

Step 3: If resulting grammar is LL(k) for some $k \geq 1$, then construct a parsing table.

5.4.5.2 Bottom-Up Parsing

In bottom-up parsing, we start with the input string and replace the terminals by respective variables such that these replacements lead to the starting symbol of the grammar. So every step takes input string close to the starting symbol. This approach is the reverse of top-down approach. It reads the input from left and uses the rightmost derivation in reverse order. Bottom-up parser is also known as shift-reduce (SR) parser. There are three actions in bottom-up parsing:

Step 1: Shift the current input (token) on the stack and read the next token.

Step 2: Reduce by some suitable LHS of production rule.

Step 3: Accept, final reduction which leads to starting symbol.

5.5 CONTEXT-FREE GRAMMAR AND LANGUAGES

A grammar $G = (V_n, T, S, P)$ is said to be a CFG if the production of G is of the form $A \rightarrow \alpha$, where $\alpha \in (V_n \cup S)^*$ and $A \in V_n$.

As we know that a CFG has no context either on left or on right, this is the reason why it is also known as context-free.

Problem 5.8: Consider a grammar $G = (V_n, T, S, P)$ having production $\{S \rightarrow aSa|bSb|x\}$. Check the production and find the language generated.

Solution: Let P_1: $S \rightarrow aSa$

P_2: $S \rightarrow bSb$

P_3: $S \rightarrow x$

(a, b, x) are terminals and S is a variable. As all the productions are of the form $A \rightarrow \alpha$, where $\alpha \in (V_n \cup S)^*$ and $A \in V_n$, hence G is a CFG, and it will produce context-free language.

Language generated: $L(G) = \{w\, x\, w^{\mathrm{R}}: w \in (a + b)^*\}$

5.5.1 Context-Free Grammar

A grammar $G = (V_n, T, S, P)$ is said to be a CFG if production $P\ \{u \rightarrow v\}$ follows these conditions:

1. $u \rightarrow v$, where $v \in (V \cup T)^*$
2. $|u| \leq |v|$ (length of u is less than v)
3. Only single variable is allowed in the left side (means u has single variable only)

Example 5.18

$$S \rightarrow aSb/\lambda$$
$$A \rightarrow aA/aB/Cb$$
$$B \rightarrow a/Da$$

5.5.2 Standard Context-Free Language

There are some standard languages under CFL such as:

1. $a^n b^n$, $a^n b^{2n}$, $a^{2n} b^n$, $a^n b^{2n+3}$, $a^{2n+2} b^{3n+5}$, etc.
2. $n_a(w) = n_b(w)$ (means number of "a" equals to number of "b" in a language), $a^m b^n$, where ($m = n$, $m > n$, $m < n$, etc.)
3. Palindrome such as [ww^{R} (even palindrome), waw^{R} (odd palindrome), etc.]

4. Generate language from grammar:
   - Grammar is $S \rightarrow aSbb|\varepsilon$
     Language generated by the above grammar is $a^n b^{2n}$.
   - Grammar is $S \rightarrow aSb|bSa|a|b$
     Language generated by the above grammar is $w\ x\ w^R$, $(x \in a, b)$.
5. Make grammar for given language:
   - Language $L = \{a^{2n}b^n\}$, where $n \geq 1$
     Grammar for given language: $S \rightarrow aaSb|\varepsilon$
   - Language $L = \{w\#w^R: w \in (a, b), \# \neq \Sigma\}$
     Grammar for a given language: $S \rightarrow aSb|bSa|\#$

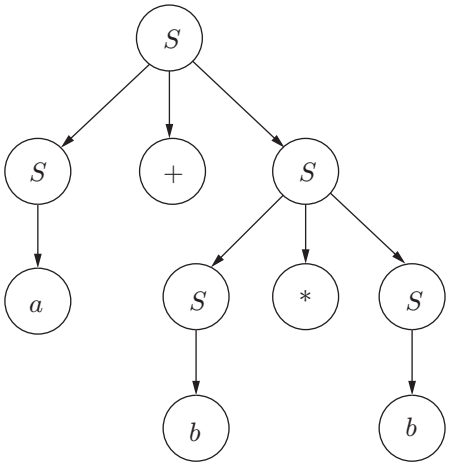
5.5.3 Derivation Tree or Parse Tree

The string generated by CFG $G = (V_n, T, S, P)$ is represented by a hierarchical structure called tree. A derivation tree or parse tree for a CFG is a tree that satisfies the following conditions:

1. If $A \rightarrow \alpha_1, \alpha_2, \alpha_3, \ldots, \alpha_n$ is a production in G, then A becomes the father of nodes labelled as $\alpha_1, \alpha_2, \alpha_3, \ldots, \alpha_n$.
2. The root has label S (starting symbol).
3. Every vertex (or node) has a label.
4. Internal nodes should be labelled with variables only.
5. The leaf nodes are labelled with ε or terminal symbols.
6. The collection of leaves from left to right yields the string w.

Problem 5.9: Consider the grammar $S \rightarrow S + S|S^*S|a|b$. Construct derivation (or parse) tree for the string $w = a + b^*b$.

Solution:



5.5.3.1 Leftmost Derivation Tree

A derivation tree is called as leftmost derivation (LMD) tree if the ordering of decomposed variable is from left to right. Thus, for generating a string $w = aab$ from grammar:

$S \rightarrow AB$ (production 1)

$A \rightarrow aaA$ (production 2)

$A \rightarrow \varepsilon$ (production 3)

$B \rightarrow bB$ (production 4)

$B \rightarrow \varepsilon$ (production 5)

LMD:

$S \rightarrow \underline{A}B$ by production 1)

$S \rightarrow aa\underline{A}B$ (by production 2)

$S \rightarrow aa\underline{B}$ (by production 3)

$S \rightarrow aab\underline{B}$ (by production 4)

$S \rightarrow aab$ (by production 5)

5.5.3.2 Rightmost Derivation Tree

A derivation tree is called rightmost derivation (RMD) tree if the ordering of decomposed variable is from right to left. Thus, for generating a string $w = aab$ from the above grammar:

RMD:

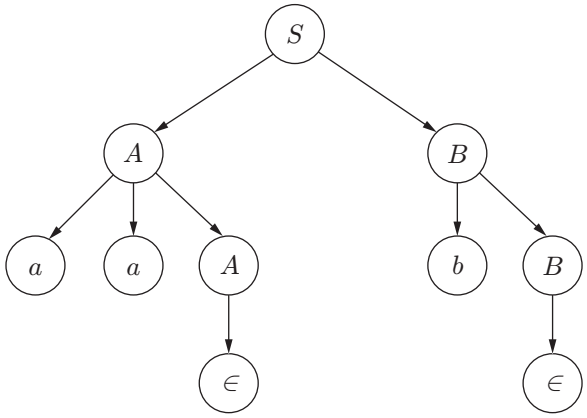
$S \rightarrow A\underline{B}$ (by production 1)

$S \rightarrow Ab\underline{B}$ (by production 4)

$S \rightarrow \underline{A}b$ (by production 5)

$S \rightarrow aa\underline{A}b$ (by production 2)

$S \rightarrow aab$ (by production 3)

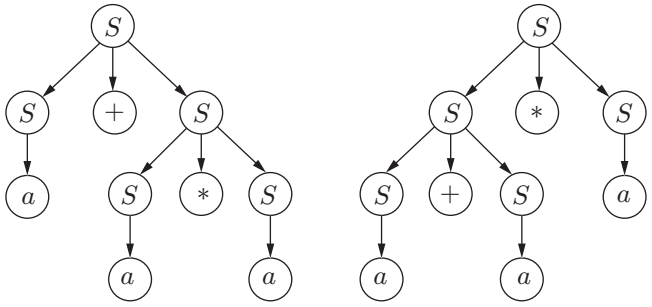


Left to right

5.5.4 Ambiguous Grammar

A grammar G is called ambiguous if for some string $w \in L(G)$, there exists two or more derivation tree (two or more LMD or two or more rightmost derivation tree). Let us consider a CFG grammar having production:

$S \rightarrow S + S|S^*S|a|b$, for string $w = a + a^*a$ have more than one LMD tree.



Note: A language (L) is called ambiguous language if and only if every grammar which generates it is ambiguous. The only known ambiguous language is $\{a^n b^m c^n\} \cup \{a^n b^m c^m\}$.

Left recursion and left factoring are the major cause for a grammar to be ambiguous. But presence of these in a grammar does not mean that grammar is ambiguous, and similarly absence of these does not mean that grammar is unambiguous.

5.5.5 Removal of Ambiguity

Ambiguities are left recursion and left factoring. In this subsection, removal of various ambiguities is discussed step by step.

5.5.5.1 Removal of Left Recursion

A production of grammar $G = (V_n, T, S, P)$ is said to be left recursive grammar if it has one of the production in a given form.

$A \rightarrow A\alpha$, where A is a variable and $\alpha \in (V_n \cup S)^*$.

Elimination of left recursion: Let the variable A have left recursive problem as follows:

$A \rightarrow A\alpha_1|A\alpha_2|A\alpha_3|...|A\alpha_n|\beta_1|\beta_2|\beta_3|...|\beta_m$, where β_1, β_2, β_3, ..., β_m do not begin with A, then we replace A production by:

$$\{A \rightarrow \beta_1 A^{|}|\beta_2 A^{|}|\beta_3 A^{|} \ ...|\beta_n A^{|}|\},$$

where $A^{|} \rightarrow \alpha_1 A^{|}|\alpha_2 A^{|}|\alpha_3 A^{|}|...|\alpha_n A^{|}|\varepsilon$

Problem 5.10: Let grammar $S \rightarrow S + S|S^*S|a|b|c$

Solution: After removing left factoring, the productions are replaced by

$$S^{|} \rightarrow + \ SS^{|}|^*SS^{|}|\varepsilon$$
$$S \rightarrow aS^{|}|bS^{|}|cS^{|}$$

5.5.5.2 Removal of Left Factoring

In grammar G, two or more production of variable A are said to have left factoring if the production are in the following form:

$$A \rightarrow \alpha\beta_1|\alpha\beta_2|\alpha\beta_3|...|\alpha\beta_m$$

where $\{\beta_1|\beta_2|\beta_3|...|\beta_m\} \in (V_n \cup S)^*$ and does not start with α. All these production have common left factor α.

Elimination of left factoring: Let variable A have (left factoring) production as follows:

$A \rightarrow \alpha\beta_1|\alpha\beta_2|\alpha\beta_3|...|\alpha\beta_m|\gamma_1|\gamma_2|\gamma_3|...|\gamma_m$, where γ_1, γ_2, γ_3, ..., γ_m and β_1, β_2, β_3, ..., β_m do not contain α as a prefix, then we replace this production by

$A \rightarrow \alpha A^{|}|\gamma_1|\gamma_2|\gamma_3|...|\gamma_m$
$A \rightarrow \beta_1|\beta_2|\beta_3|...|\beta_m$

Problem 5.11: Let grammar $A \rightarrow abc|abd|abe$ remove left factoring.

Solution: After removing left factoring, the productions are replaced by

$A \rightarrow abA^{|}$
$A \rightarrow c|d|e$

5.5.6 Context-Free Grammar Simplification

For simplification of context-free language, it is necessary to eliminate variable, terminal and production having the following properties:

1. Unit production ($A \rightarrow B$)
2. Null production ($A \rightarrow \lambda$)
3. Useless production, a variable which does not occur even in a single derivation

5.5.6.1 Removal of Unit Production

A production (P) in CFG is said to be a unit production if it is in the following form:

$A \rightarrow B$ where $A, B \in V$ (variable). There are three steps to remove a unit production.

Step 1: Find $\widehat{P} = P -$ (unit production)

Step 2: Find unit production and draw a unit production dependency graph.

Step 3: Add new rules to the grammar.

Problem 5.12: Remove the unit production from the given CFG grammar.

$$S \rightarrow Aa|B$$
$$B \rightarrow A|bb$$
$$A \rightarrow a|bc|B$$

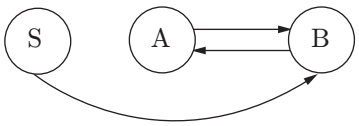
Solution:

Step 1: Find a non-unit production $(\widehat{P}) = P -$ (unit production)

$$S \rightarrow Aa$$
$$B \rightarrow bb$$
$$A \rightarrow a|bc$$

Step 2: Find a unit production and draw a unit production dependency graph.

$$S \rightarrow B$$
$$B \rightarrow A$$
$$A \rightarrow B$$



Step 3: Add new rules to the grammar. Check that from each variable through other variables on which terminals we can reach.

$$S \rightarrow a|bc|bb$$
$$A \rightarrow bb$$
$$B \rightarrow a|bc$$

Final grammar after removing a unit production is as follows:

$$S \rightarrow Aa|a|bc|bb$$
$$A \rightarrow a|bc|bb$$
$$B \rightarrow bb|a|bc$$

Note: Removal of a unit production has made B and the associated production useless.

5.5.6.2 Removal of Null Production

$A \rightarrow \lambda$ is a null production and A is a nullable variable; λ should be removed because it creates difficulties for the compilers. There are two steps to remove a null production.

Step 1: Identify the nullable variable.

Step 2: Find $\widehat{P}$ production without null.

Problem 5.13: Remove null (λ) production from the given CFG grammar.

$$S \rightarrow ABaC$$
$$A \rightarrow BC$$
$$B \rightarrow b|\lambda$$
$$C \rightarrow D|\lambda$$
$$D \rightarrow d$$

Solution:

Step 1: Identify nullable variables.

Variables B and C directly produce null $N = \{B, C\}$. But variable A is related to B and C, so A will also produce a null production.

$$N = \{A, B, C\}$$

Step 2: Find $\widehat{P}$ production without null.

$$S \rightarrow ABaC$$
$$A \rightarrow BC$$
$$B \rightarrow b$$
$$C \rightarrow D$$
$$D \rightarrow d$$

Generate grammar without null production ($G^{|}$) by putting null value to nullable variables one by one in production without null. For example, put $A = \lambda$ in $S \rightarrow ABaC$, we will get $S \rightarrow BaC$. Do this process for all nullable variables.

$$S \rightarrow ABaC|BaC|AaC|ABa|aC|aB|Aa|a$$
$$A \rightarrow BC|B|C$$
$$B \rightarrow b$$
$$C \rightarrow D$$
$$D \rightarrow d$$

5.5.6.3 Removal of Useless Production

Let $G = (V_n, T, S, P)$ be a CFG. A variable $A \in V$ is said to be useless if there is not even a single derivation which uses that production.

Example 5.19

$$S \rightarrow A$$
$$A \rightarrow aA|a$$
$$B \rightarrow bA$$

The variable B is useless and so is the production $B \rightarrow bA$. Although b can drive a terminal string, there is no way to reach on B from the starting variable S.

Problem 5.14: Consider a given grammar and eliminate useless production.

$$S \rightarrow aS|A|C$$
$$A \rightarrow a$$
$$B \rightarrow aa$$
$$C \rightarrow aCb$$

Solution:

Step 1: First find out those variables which lead to terminal strings. Because $A \rightarrow a$ and $B \rightarrow aa$, variables A and B directly come under this set. But due to the dependency on A, variable S also generates terminal string $S \rightarrow A \rightarrow a$. So,

$$S \rightarrow aS|A$$
$$A \rightarrow a$$
$$B \rightarrow aa$$

Step 2: Now we have to find those variables which cannot be reached from the starting variable. B is the only variable which cannot be reached from S. So B is a useless variable. Grammar after eliminating useless production and variables is as follows:

$$S \rightarrow aS|A$$
$$A \rightarrow a$$

5.5.7 Chomsky Normal Form

Chomsky normal form (CNF) has restriction on the length of the right-hand side of a production, either two variables or single terminal is allowed in the right side of a production such as $S \rightarrow AB$, $S \rightarrow a$.

Problem 5.15: Convert the given grammar into CNF form:

$$S \rightarrow aAD$$
$$A \rightarrow aB|bAB$$
$$B \rightarrow b$$
$$D \rightarrow d$$

Solution:

$S \rightarrow aAD$ or $\{S \rightarrow XD,\ X \rightarrow YA,\ Y \rightarrow a\}$

$A \rightarrow aB$ or $\{A \rightarrow ZB,\ Z \rightarrow a\}$

$A \rightarrow bAB$ or $\{A \rightarrow KB,\ K \rightarrow LA,\ L \rightarrow b\}$

This grammar can be written as in CNF $G^{|}$ as

$V_n = \{S,\ A,\ B,\ D,\ K,\ L,\ X,\ Y,\ Z\}$

$\Sigma = \{a,\ b,\ d\}$

$P^{|} = \{S \rightarrow XD,\ X \rightarrow YA,\ Y \rightarrow a,\ A \rightarrow ZB,\ Z \rightarrow a,$
$A \rightarrow KB,\ K \rightarrow LA,\ L \rightarrow b\}$

5.5.8 Greibach Normal Form

A CFG is called in Greibach normal form (GNF) if all productions have the form $\{A \rightarrow ax$ or $A \rightarrow a\}$, where $a \in T$ and $x \in V^*$. Means RHS of production are either a terminal or a terminal followed by variables.

Problem 5.16: Convert the given grammar into GNF

$$S \rightarrow AB$$
$$A \rightarrow aA|bB|b$$
$$B \rightarrow b$$

Solutions: The GNF of the given grammar is

$$S \rightarrow aAB|bBB|bB$$
$$A \rightarrow aA|bB|b$$
$$B \rightarrow b$$

5.5.9 Identification of Language

1. If language is finite then it will surely be a regular language. Example: Let $L = \{a^m b^n,\ m + n = 10\}$ as m and n are finite, so it is a regular language. But it does not mean that if a language is infinite then it cannot be regular such as (a^*b^*) is an infinite and regular language.
2. If there is a comparison between m and n then it will be CFL. Example: Let $L = \{a^m b^n\ (m \geq n)\}$.
3. Let language $L = \{a^m b^n\}$, and if m or n is nonlinear then it will not be accepted by PDA, so it will fall in the category of CSL.
4. If there are more than one comparison then it will be CSL. Example: Language $L = \{a^m b^n c^o,\ m \geq n$ and $o \geq n\}$.
5. All modular machines are regular language.
6. All palindrome languages are CFL language.

5.6 TURING MACHINE

In 1936, an English mathematician Alan Turing suggested the concept of Turing machine (TM). This machine can simulate the behaviour of a general purpose

computer. A TM is a generalization of a PDA, which uses a tape instead of tape and stack. The length of the tape is assumed to be infinite. A tape is divided into cells, and one cell can hold one input symbol. The head of the TM is capable to read and write on the tape and can move left or right or remain static.

TMs accept the languages defined by type 1 grammar, and these languages are called recursively enumerable or type 1 languages.

5.6.1 Model of a Turing Machine

A TM consists of a tape which is finite at the left end and infinite at the right end and has a finite control and read/write head (Fig. 5.6).

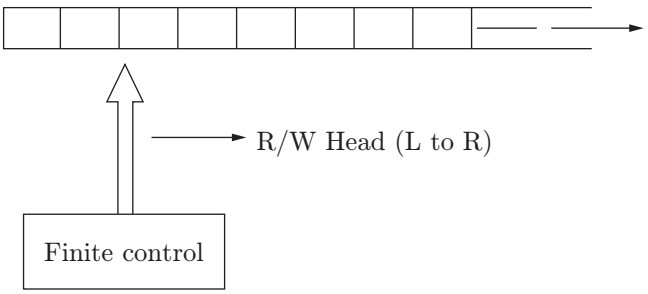


Figure 5.6 | Model of a turing machine.

5.6.1.1 Mathematical Description of a Turing Machine

A TM can be described by a seven-tuple set $(Q, \Sigma, \Gamma, \delta, q_0, \#, F)$, where

1. Q is the finite set of states (not including the halt state (h)).
2. Σ is the input alphabet which is a subset of the tape alphabet (not including the blank symbol $\#$).
3. Γ is the finite set of symbols called the tape alphabet.
4. δ is the transition function which maps from $Q \times \Gamma \rightarrow Q \times \Gamma \times [L, R]$.
5. $\# \in \Gamma$ is a special symbol called blank.
6. $q_0 \in Q$ is the initial state.
7. $F \subseteq Q$ is the set of final states.

A TM can be deterministic or non-deterministic depending on the number of moves in a transition. If a TM has at the most one move in a transition, then it is a deterministic Turing machine (DTM). If there is more than one move, then it is a non-deterministic Turing machine (NTM). A standard TM reads one cell of the tape and changes its state (optional) and moves left or right by one cell.

The transition function (δ) is defined as $\delta(p, a) = (q, b, D)$, where p is the present state, q is the next state a, $b \in \Gamma$ and D is the movement {(left (L) or right (R)}.

After reading an input $a \in \Sigma$, TM does one of the following:

1. Replaces a by b and moves right (R) as $\delta(p, a) = (q, b, R)$.
2. Without write, anything moves right (R) as $\delta(p, a) = (q, a, R)$.
3. Replace a by b and move left (L) as $\delta(p, a) = (q, b, L)$.
4. Without write, anything moves left (L) as $\delta(p, a) = (q, a, L)$.

The change of state in the above transition is optional (means may or may not change).

5.6.1.2 Language Reorganization by a Turing Machine

A TM can be used as a language recognizer. It recognizes all languages (regular, CFL, CSL, type 0).

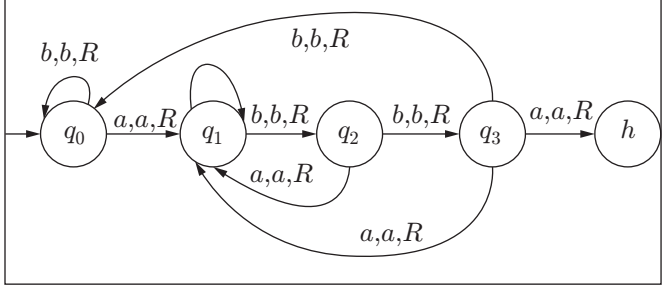
Let w be a string to be written on tap. The Turing machine is started from initial state q_0, read write head is placed on the left most symbol of w. with the sequence of moves, if TM enters to a final state and halts then string w is said to be accepted by Turing machine.

Problem 5.17: Design a TM for $L = \{w: w \in (a + b)^*$ ending in substring $abb\}$

Solution: Let TM $M = \{(q_0, q_1, q_2, q_3, h), (a, b), (a, b, \#), \delta, q_0, \#, h\}$ accepts language L, where δ is defined as follows:

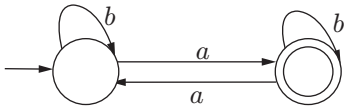
$\delta(q_0, b) = (q_0, b, R)$
$\delta(q_0, a) = (q_1, a, R)$
$\delta(q_1, a) = (q_1, a, R)$
$\delta(q_1, b) = (q_2, b, R)$
$\delta(q_2, a) = (q_1, a, R)$
$\delta(q_2, b) = (q_3, b, R)$
$\delta(q_3, a) = (q_1, a, R)$
$\delta(q_3, b) = (q_0, b, R)$
$\delta(q_3, \#) = (h, \#, S)$

The transition diagram is shown in the following figure.

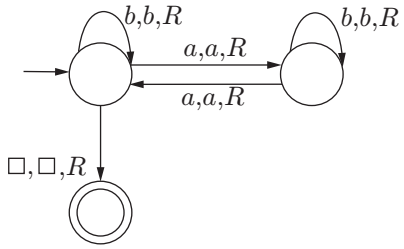


5.6.2 Designing a Turing Machine

Let us consider a problem for which we will design a TM. Suppose we have to design a TM which will halt on a final state if a string has an even number of "a"s'. So, first we will design a DFA which accepts a string having an even number of "a"s', and after that we will design a TM for the same problem.



This regular machine will accept only those strings which have an even number of a's. Now we will make a final state as non-final and add one final state on blank input.

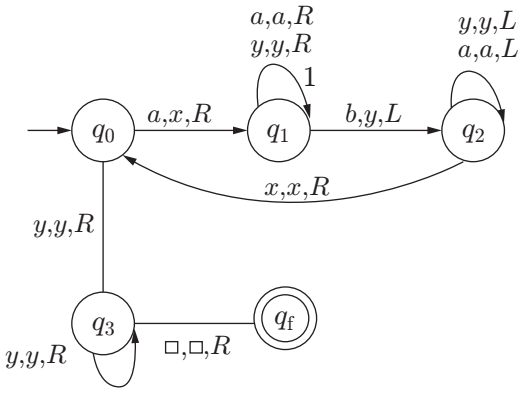


Problem 5.18: Design a TM which accepts a language $L = \{a^n b^n,\ n \geq 1\}$.

Solution: First make a transition function move based on an input in the tape which is shown in the figure.

$\delta(q_0,\ a) = (q_1,\ x,\ R)$
$\delta(q_1,\ a) = (q_1,\ a,\ R)$
$\delta(q_1,\ y) = (q_1,\ y,\ R)$
$\delta(q_1,\ b) = (q_2,\ y,\ L)$
$\delta(q_2,\ y) = (q_2,\ y,\ L)$
$\delta(q_2,\ a) = (q_2,\ a,\ L)$
$\delta(q_2,\ x) = (q_0,\ x,\ R)$
$\delta(q_0,\ y) = (q_3,\ y,\ R)$
$\delta(q_3,\ y) = (q_3,\ y,\ R)$
$\delta(q_3,\ \#) = (q_f,\ \#,\ R)$

A TM for a given language $L = \{a^n b^n,\ n \geq 1\}$.



5.6.3 Recursive and Recursive Enumerable Languages

The properties of recursive and recursive enumerable languages are as follows:

1. A language L is recursive enumerable (RE) if and only if there exists a TM that accepts it, machine must halt and accept $\forall\ w \in L$.
2. A language L is said to be recursive (REC) if and only if there exists a TM that accepts it and which halts on $\forall\ w \in \Sigma^*$.
3. REC is a decidable language.
4. RE is a semi-decidable language.
5. Both RE and REC have enumerable procedure (EP). A procedure is called an EP iff it lists all members of L in a finite amount of time.
6. Only REC has membership algorithm.
7. A set S is countable if it has an EP.
8. If A and B are countable then $(A \times B)$ is also countable.
9. Real numbers are uncountable infinite.
10. If a language L is RE then L has an EP but $\overline{L}$ does not have an EP.
11. If a language L is REC then both L and $\overline{L}$ have an EP. Every subset of countable set is countable, so every language L is countable because $L \subseteq \Sigma^*$ which is countable. It means every language has an EP but not those languages which are "not RE" or outside RE.
12. Set of all TM is countable.
13. If L is REC then $\overline{L}$ also will be REC.
14. If L is RE then $\overline{L}$ may or may not be RE.
15. If $\overline{L}$ is RE then L may or may not be RE.
16. Languages (regular, CFL, CSL, REC, RE) are countable infinite.
17. If both L and $\overline{L}$ is Turing enumerable then they are REC.

5.6.4 Variation of Turing Machine

Different variants of Turing machines are available. These variations are given as follows:

1. Multi-track Turing machine
2. Multi-tape Turing machine
3. Non-deterministic Turing machine
4. Universal Turing machine
5. Offline Turing machine
6. Turing machine with semi-infinite machine

5.7 CLOSURE AND DECIDABILITY

There are different closure and decidability properties for various languages. Tables 5.6 and 5.7 describe these properties for different languages.

Table 5.6 | Closure property of languages

Operation	Languages					
	Regular	DCFL	CFL	CSL	REC	RE
$\cup$	✓	×	✓	✓	✓	✓
$\cap$	✓	×	×	✓	✓	✓
L^c	✓	✓	×	×	✓	×
-	✓	×	✓	✓	✓	✓
*	✓	×	✓	✓	✓	✓
$L_1 \cup R$	✓	×	✓	✓	✓	✓
$L_1 \cap R$	✓	✓	✓	✓	✓	✓
L^R	✓	×	✓	–	–	–
$h(L)$	✓	×	✓	–	–	–
$h^{-1}(L)$	✓	✓	✓	–	–	–
L_1/L_2	✓	✓	–	–	–	–

✓{Surely exists}

× {May or may not exist}

– {Not known}

Table 5.6 is interpreted as follows:

1. {$\cup$} means that if there are two languages L_1 and L_2 which are regular then their union will be regular, whereas if languages are DCFL then their union will not be DCFL.
2. {$L_1 \cup R$} means that from two languages if one is regular (R) and the other is different, then their union will be result in a language similar to the second language. For example, if one language is regular and the other is DCFL, then their union will be a DCFL.
3. L^R represents reverse of a language; if there are two CFLs L_1 and L_2 then their union will be a CFL and their intersection will not be a CFL.
4. (.) shows concatenation operation.
5. (*) denotes multiplication operation.

Table 5.7 | Decidability table for languages

Algorithm Exists	Languages				
	Regular	CFL	CSL	REC	RE
Membership: $w \in \Sigma^*$, L then $w \in L$?	✓	✓	✓	✓	✓
Finiteness: Given language L is finite or infinite?	✓	✓	×	×	×
Emptiness: Language L is empty or not?	✓	✓	×	×	×
Equivalence: Two languages L_1 and L_2 are equal or not?	✓	×	×	×	×
Ambiguity: Language L is ambiguous or not?	✓	×	×	×	×
Regularity: Language is regular or not?	✓	×	×	×	×
Disjointness: Two language intersection is empty or not?	✓	×	×	×	×
Everything: Language L, then $L = \Sigma^*$	✓	×	×	×	×

✓ {Denotes that algorithm exists to decide about the question (decidable)}

× {Denotes that algorithm does not exist about that question (means undecidable)}

IMPORTANT FORMULAS

1. Number of states in a machine which accept m a' s and n b' s:
$$[(m + 1)^*(n + 1) + 1]$$
2. Mod n machines have n states.
3. Mod machine have no trap states.
4. If a machine accepts string length exactly n, then it has $(n + 2)$ states.
5. If a machine accepts string length $\leq n$, then it has $(n + 2)$ states.
6. If a machine accepts string length $\geq n$, then it has $(n + 1)$ states.
7. m States, n outputs Mealy machine $\equiv$ Moore machine containing $\leq mn + 1$ states.
8. m States, n outputs Moore machine $\equiv$ Mealy machine containing $\leq m$ states.
9. Output length of Mealy machine is equal to input length.
10. Output length of Moore machine is one greater than the input length because first output symbol is additional without reading any symbol from the input.
11. If language is finite then it will surely be regular. Example: Let $L = \{a^m b^n,\ m + n = 10\}$ as m and n are finite so it is a regular language. But it does not mean that if a language is infinite then it cannot be regular such as (a^*b^*) is an infinite and regular language.
12. If there is a comparison between m and n then it will be a CFL. Example: Let $L = \{a^m b^n,\ (m \geq n)\}$.
13. Let language $L = \{a^m b^n\}$ and if m or n is non-linear then it will not be accepted by PDA, so it will fall in the category of CSL.
14. If there are more than one comparison then it will be a CSL. Example: Language $L = \{a^m b^n c^o,\ m \geq n$ and $o \geq n\}$.
15. All modular machines are regular language.
16. All palindrome languages are CFL language.

Variation of NFA/DFA

17. DFA/NFA + (left $\leftrightarrow$ right) move $\equiv$ (2 − DFA)/(2 − NFA) means whatever a 2 − DFA can do, that can be done by simple DFA with left – right move.
18. DFA/NFA + (left $\leftrightarrow$ right) move + (read/write) head $\equiv$ (2 − DFA)/(2 − NFA).
19. DFA + 1 stack $\equiv$ DPDA).
20. NFA + 1 stack $\equiv$ NPDA).
21. DFA + 2 stack $\equiv$ TM.
22. DFA/NFA + 2 counter $\equiv$ TM.
23. DFA/NFA + 2 stack $\equiv$ DFA/NFA + 2 counter.
24. Power of {(DFA/NFA) < (DFA/NFA + 1 counter) < (DFA/NFA + 1 stack)}
25. {(DFA/NFA + 1 counter) < (DFA/NFA + 2 counter)}.

SOLVED EXAMPLES

1. Which of the following is a regular expression?

   (a) $L = \{0^m 1^n \mid m, n \geq 0\}$
   (b) $L = \{0^n 1^n \mid n \geq 0\}$
   (c) $L = \{xx \mid x \in \{0, 1\}^*\}$
   (d) $L = \{1^{n^2} 0^n \mid n \geq 0\}$

   Solution: In option (a), the given expression will generate any number of 0's followed by any number of 1's, which can be accepted by FA. So, it is a regular expression.

   In option (b), the given expression will generate number of 0's followed by number of 1's (both should be same in number), which will be accepted by PDA. So, it is not a regular expression.

   In option (c), the given expression will generate any string "x" twice, which will be accepted by PDA. So it is not a regular expression.

   In option (d), the given expression will generate twice the n number of 1's followed by n number of 0's, which cannot be accepted by FA. So, it is not a regular expression.

   Ans. (a)

2. The class of context-free language is not closed under

   (a) union
   (b) star
   (c) repeated concatenation
   (d) intersection

Solution: Context-free language is not close under intersection. Please refer table 5.6 (closure property of languages) in text.

Ans. (d)

3. Which of the following is true for regular sets $A = (1101 + 1)^*$ and $((1101)^*1^*)^*$?

(a) $A \subset B$ (b) $B \subset A$
(c) $A = B$ (d) A and B are incomparable

Solution: Both the regular expressions will generate the same set of strings, so both the regular sets are equal.

Ans. (c)

4. The language generated by the following grammar is $S \rightarrow aSa|bSb|\varepsilon$

(a) $a^m b^n$ $n \geq 0$, $m \geq 0$
(b) $a^n b^m$ $n \geq 1$, $m \geq 1$
(c) Odd length palindrome
(d) Even length palindrome

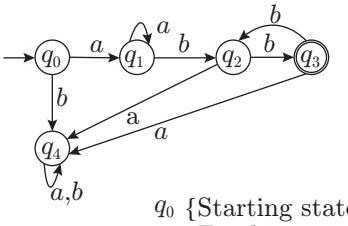
Solution: In the given grammar, variable S will generate a pair of symbol 'a' or symbol 'b' with variable S in the middle of them, and this is in loop so it can be generated any number of times. In the last variable, S will be replaced by ε. Some of the strings which can be generated by the given grammar are {aa, bb, $abba$, $aabbaa$, $baab$, etc}. So, the given grammar will generate even length palindrome.

Ans. (d)

5. Number of states in DFA accepting the following language is $L = \{a^n b^{2m} | n, m \geq 1\}$

(a) 2 (b) n
(c) m (d) 5

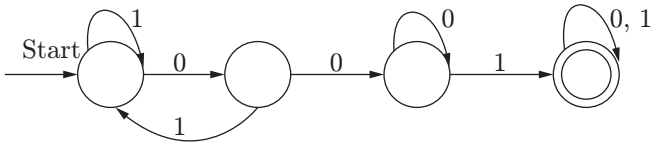
Solution: DFA for given language $L = a^n b^{2m} | n$, $m \geq 1$ is



q_0 {Starting state}
q_3 {Final state}

Ans. (d)

6. Consider the following DFS automaton M.



Let S denote the seven bits binary strings in which the first, fourth and last bits are 1. The number of strings in S that are accepted by M is

(a) 6 (b) 5
(c) 4 (d) 7

Solution: Strings which will be accepted by machine M can be found by fixing 1 at places first, fourth and last. We will have following four strings:

1001001 1001011 1001111 1111001

Ans. (c)

7. Which of the following CFG will generate the language $L = \{a^m b^n c^p d^q \mid m + n = p + q\}$

(a) $S \rightarrow aSd|A|B$
$A \rightarrow aAc|C$
$B \rightarrow bBd|D$
$C \rightarrow bCc|\varepsilon$

(b) $S \rightarrow aSd|A|B$
$A \rightarrow aAc|D$
$B \rightarrow bBd|C$
$C \rightarrow CBc|\varepsilon$

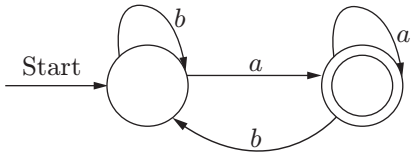
(c) $S \rightarrow aSd|A|B$
$A \rightarrow aAB|C$
$B \rightarrow bBd|C$
$C \rightarrow bCc|\varepsilon$

(d) $S \rightarrow aSd|A|B$
$A \rightarrow aAc|C$
$B \rightarrow bBd|C$
$C \rightarrow bCc$

Solution: Strings generated by L should have sum of number of 'a' and 'b' equal to total number of 'c' and 'd'. Only in option (c), the grammar can generate such string which has number of $(a + b)$ equal to number of $(c + d)$.

Ans. (c)

8. Consider the FA shown in the figure below, which language is accepted by the FA



(a) $b + (a + b)^*b$ (b) $(b + a + b)^*a$
(c) $b + a^*b$ (d) $b + a^*b^*$

Solution: Given that FA will accept any string which has the last symbol 'a'. So, only option (b) expression $(b + a + b)^*$ a can generate all combinations of strings which end with symbol 'a'.

Ans. (b)

9. Let $L = L_1 \cap L_2$, where L_1 and L_2 are languages defined below

$L_1 = \{a^m b^m c a^n b^m \mid m, n \geq 0\}$ and $L_2 = \{a^i b^j c^k \mid i, j, k \geq 0\}$. Then L is

(a) regular but not recursive
(b) context-free
(c) context-free but not regular
(d) recursively enumerable but not context-free

Solution: Language L_1 is CFL and language L_2 is regular. So, the result of $L_1 \cap L_2$ will be context free.

Ans. (c)

10. Which of the following is/are correct?

I. A language is context-free if and only if it is accepted by the PDA.
II. PDA is a finite automata with push-down stack.

(a) Only I is true
(b) Only II is true
(c) Both I and II are true
(d) Both I and II are false

Solution: Both I and II are true.

Ans. (c)

11. How many strings of length less than 4 contains the language described by the regular expression $(x + y)^*y(a + ab)^*$?

(a) 3 (b) 9
(c) 10 (d) 11

Solution: From the regular expression $(x + y)^* y(a + ab)^*$, it can be seen that it contains 11 strings as follows:

$\{xy, yy, y, ya, yab, xya, yya, yaa, xxy, yyy, xyy\}$

Ans. (d)

12. Consider the following laws:

L_1: $(L^*)^* = L^*$
L_2: $\varnothing^* = \varnothing$
L_3: $\varepsilon^* = \varepsilon$
L_4: $L^+ = L^* + \varepsilon$
L_5: $L^* = \varepsilon + L^+$

Which of the above laws hold for regular expression?

(a) L_1, L_2 and L_5 (b) L_1, L_3 and L_5
(c) L_2, L_3 and L_4 (d) L_1, L_4 and L_5

Solution:

L_1: both the expression generates any string made up of input symbol of L. it also include ε.

L_2: $\varnothing^*$ can generate $\{\varnothing$ and $\varepsilon\}$

L_3: ε^* can only generate ε.

L_4: L^+ contain all the string made up of input symbol of L excluding ε.

L_5: L^* contain all the string made up of input symbol of L including ε.

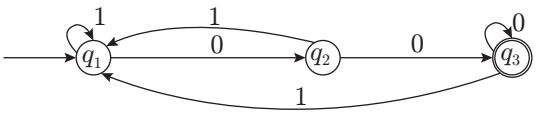
Son only L_1, L_3 and L_5 are applicable.

Ans. (b)

13. How many states does the DFA constructed for the set of all strings ending with "00" have?

(a) 1 (b) 3
(c) 2 (d) 2^2

Solution: DFA will construct as:



Ans. (b)

14. Which of the following CFG cannot be simulated by an FSM?

(a) $S \rightarrow Sa|b$
(b) $S \rightarrow aSb|ab$
(c) $S \rightarrow abX$, $X \rightarrow cY$, $Y \rightarrow d|aX$
(d) None of these

Solution: Given Grammar $S \rightarrow aSb|ab$ generates language $\{a^n b^n\}$ means language contains equal number of a's and b's. So, it cannot be simulated by finite state machine.

Ans. (b)

15. Let $r = 1(1 + 0)^*$, $s = 11^*0$ and $t = 1^*0$ be three regular expressions. Which one of the following is true?

(a) $L(s) \subseteq L(r)$ and $L(s) \subseteq L(t)$
(b) $L(r) \subseteq L(s)$ and $L(s) \subseteq L(t)$
(c) $L(s) \subseteq L(t)$ and $L(s) \subseteq L(r)$
(d) $L(t) \subseteq L(s)$ and $L(s) \subseteq L(r)$

Solution:

$r = 1(1 + 0)^*$, the language corresponds to r having all strings starting with 1.
$s = 11^*0$, the language corresponds to s having all strings starting with 1 followed by any number of 1 and ending with 0.
$t = 1^*0$, the language corresponds to t having all strings ending with 0.
So, $L(s) \subseteq L(r)$ and $L(s) \subseteq L(t)$. Therefore, option (a) is correct.

Ans. (a)

16. Which two of the following four regular expressions are equivalent?

(i) $(00)^*(\varepsilon + 0)$ (ii) $(00)^*$
(iii) 0^* (iv) $0(00)^*$

(a) (i) and (ii) (b) (ii) and (iii)
(c) (i) and (iii) (d) (iii) and (iv)

Solution:

(i) $(00)^*(\varepsilon + 0)$ represents any number of 0's
(ii) $(00)^*$ represents even no. of 0's
(iii) 0^* represents any number of 0's
(iv) $0(00)^*$ represents odd number of 0's

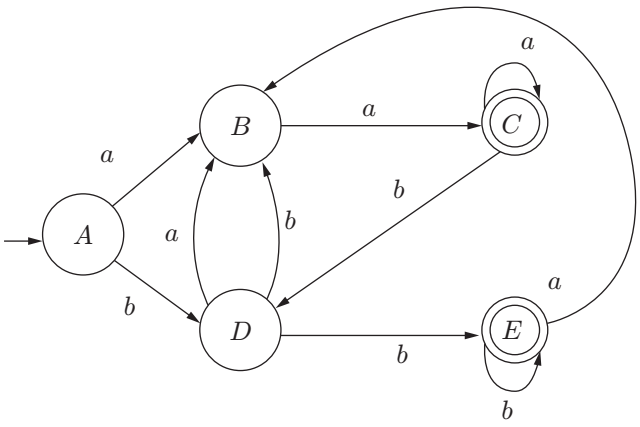
So, (i) and (iii) are the same expressions.

Ans. (c)

17. Let L be the set of all binary strings whose last two symbols are the same. The number of states in the minimum state deterministic finite state automation accepting L is

(a) 2 (b) 5
(c) 8 (d) 3

Solution: Minimum number of states in DFA which accept all binary strings whose last two symbols are the same is 5.



Ans. (b)

18. Consider the languages L_1, L_2, L_3 as given below:

$L_1 = \{1^p 0^q | p, q \in N\}$
$L_2 = \{1^p 0^q | p, q \in N \text{ and } p = q\}$
$L_3 = \{1^p 0^q | p, q, r \in N \text{ and } p = q = r\}$

Which of the following statements is NOT TRUE?

(a) Pushdown Automata (PDA) can be used to recognize L_1 and L_2.
(b) L_1 is a regular language.
(c) All the three languages are context-free.
(d) Turing machine can be used to recognize all the three languages.

Solution: Language L_1 is regular because it can be accept by NFA. Language L_2 and L_3 are CFL because they have comparison in between p and q.

Ans. (c)

19. Which of the following language over $\{a, b, c\}$ is accepted by a deterministic pushdown automata?

(a) $wcw^R \mid w \in \{a, b\}^*\}$
(b) $ww^R \mid w \in \{a, b, c\}^*\}$
(c) $\{a^n b^n c^n \mid n \geq 0\}$
(d) $\{w \mid w$ is a palindrome over $\{a, b, c\}$

Solution: The language $L = wcw^R \mid w \in \{a, b\}^*\}$ is accepted by DPDA because it is the only one which is DCFL. For the above language everything is pushed into the stack until "c" appears. When "c" appears all elements are removed from the stack if they match every symbol in w^R.

Ans. (a)

20. Which one of the following is the strongest correct statement about a finite language over some finite alphabet $\sum$?

(a) It could be undecidable.
(b) It is Turing machine recognizable.
(c) It is a regular language.
(d) It is a context-sensitive language.

Solution: Every finite language is regular and also CFL, CSL, RE (Turing machine recognizable) because regular language $\subset$ CFL $\subset$ CSL $\subset$ RE. So, the strongest correct statement is option (c).

Ans. (c)

GATE PREVIOUS YEARS' QUESTIONS

1. Ram and Shyam have been asked to show that a certain problem Π is NP-complete. Ram shows a polynomial time reduction from the 3-SAT problem to Π, and Shyam shows a polynomial time reduction from Π to 3-SAT. Which of the following can be inferred from these reductions?

(a) Π is NP-hard but not NP-complete.
(b) Π is in NP, but is not NP-complete.
(c) Π is NP-complete.
(d) Π is neither NP-hard nor in NP.

(GATE 2003: 1 Mark)

Solution: Reduction from the 3-SAT problem to Π is NP hard and reduction from Π to 3-SAT shows that Π is in NP.

As Π is in NP and it is NP hard, so it can be inferred that Π is NP complete.

Ans. (c)

2. If strings of a language L can be effectively enumerated in lexicographic (i.e., alphabetic) order, which of the following statements is true?

(a) L is necessarily finite.
(b) L is regular but not necessarily finite.

(c) L is context-free but not necessarily regular.
(d) L is recursive but not necessarily context-free.

(GATE 2003: 1 Mark)

Solution: L is recursive because recursive languages can be effectively enumerated in lexicographic order.

Ans. (d)

3. Consider the set $\sum^*$ of all strings over the alphabet $\sum = \{0, 1\}$. $\sum^*$ with the concatenation operator for strings

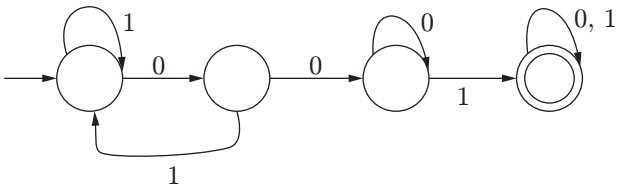
(a) does not form a group
(b) forms a non-commutative group
(c) does not have a right identity element
(d) forms a group if the empty string is removed from $\sum^*$

(GATE 2003: 1 Mark)

Solution: Fact

Ans. (a)

4. Consider the following deterministic finite state automaton M.



Let S denote the set of seven bit binary strings in which the first, the fourth, and the last bits are 1. The number of strings in S that are accepted by M is

(a) 1 (b) 5
(c) 7 (d) 8

(GATE 2003, 2 Marks)

Solution: Strings that are accepted are: 1001001, 1001011, 1001101, 1001111, 1111001, 1101001, and 1011001.

Ans. (c)

5. Let $G = (\{S\}, (a, b\}R, S)$ be a context-free grammar where the rule set R is

$$S \rightarrow a\,S\,b \mid S\ S \mid \varepsilon$$

Which of the following statements is true?

(a) G is not ambiguous.
(b) There exists $x, y, \in L(G)$ such that $xy \notin L(G)$.
(c) There is a deterministic pushdown automaton that accepts $L(G)$.
(d) We can find a deterministic finite state automaton that accepts $L(G)$.

(GATE 2003: 2 Marks)

Solution: Option (a) is False. G is ambiguous due to the possibility of two parsing trees for the same string.

Option (b) is True. Let $x = ab$ and $y = ba$, so $abba$ does not belong to $L(G)$.

Ans. (b)

6. A single tape Turing machine M has two states q_0 and q_1, of which q_0 is the starting state. The tape alphabet of M is $\{0, 1, B\}$ and its input alphabet is $\{0, 1\}$. The symbol B is the blank symbol used to indicate end of an input string. The transition function of M is described in the following table

	0	1	B
q_0	q_1, 1, R	q_1, 1, R	Halt
q_1	q_1, 1, R	q_0, 1, L	q_0, B, L

The table is interpreted as illustrated below.

The entry (q_1, 1, R) in row q_0 and column 1 signifies that if M is in state q_0 and reads 1 on the current tape square, then it writes 1 on the same tape square, moves its tape head one position to the right and transitions to state q_1.

Which of the following statements is true about M?

(a) M does not halt on any string in $(0 + 1)^+$.
(b) M does not halt on any string in $(00 + 1)^*$.
(c) M halts on all string ending in a 0.
(d) M halts on all string ending in a 1.

(GATE 2003: 2 Marks)

Solution: M does not halt on any string in $(0+1)^+$.

Ans. (a)

7. Define languages L_0 and L_1 as follows:

$L_0 = (<M, w, 0> | M \text{ halts on } w)$
$L_1 = \{<M, w, 1> | M \text{ does not halt on } w\}$

Here $<M, w, i>$ is a triplet, whose first component, M, is an encoding of a Turing machine, second component, w, is a string and third component, i, is a bit. Let $L = L_0 \cup L_1$. Which of the following is true?

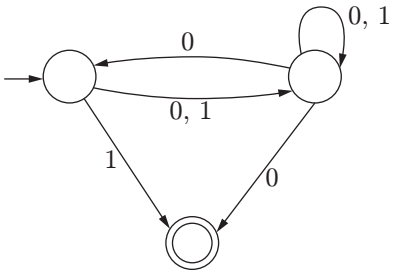
(a) L is recursively enumerable, but $\overline{L}$ is not.
(b) $\overline{L}$ is recursively enumerable, but L is not.
(c) Both L and $\overline{L}$ are recursive.
(d) Neither L nor $\overline{L}$ is recursively enumerable.

(GATE 2003: 2 Marks)

Solution: L contains strings that halts on w and does not halt on w. L can be recursively enumerable if it contains valid strings. So, L is not recursively enumerable, but $\overline{L}$ is recursively enumerable.

Ans. (b)

8. Consider the NFA M shown below.

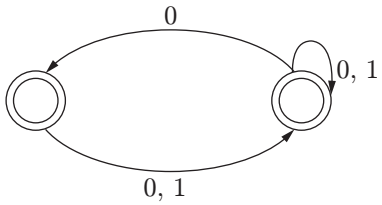


Let the language accepted by M be L. Let L_1 be the language accepted by the NFA M_1 obtained by changing the accepting state of M to a non-accepting state and by changing the non-accepting states of M to accepting states. Which of the following statements is true?

(a) $L_1 = \{0,1\}^* - L$ (b) $L_1 = \{0,1\}^*$
(c) $L_1 \subseteq L$ (d) $L_1 = L$

(GATE 2003: 2 Marks)

Solution:



Strings accepted by resulting NFA: $\{\in,0,1,00,01,10,11....\} = (0+1)^*$

Ans. (b)

9. The problems 3-SAT and 2-SAT are

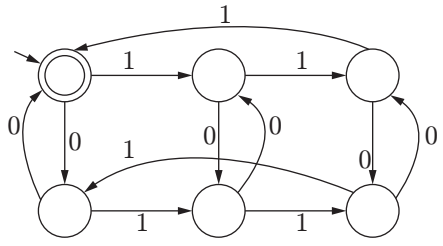
(a) both in P.
(b) both NP-complete
(c) NP-complete and in P, respectively
(d) undecidable and NP-complete, respectively

(GATE 2004: 1 Mark)

Solution: 3-SAT are NP-complete problems. 2-SAT are P problems.

Ans. (c)

10. The following finite state machine accepts all those binary strings in which the number of 1's and 0's are, respectively,



(a) divisible by 3 and 2
(b) odd and even
(c) even and odd
(d) eivisible by 2 and 3

(GATE 2004: 2 Marks)

Solution: The machine accepts strings that have 1's divisible by 3 and 0's divisible by 2. For example, strings 11100, 10011, 00111 will be accepted by this machine and strings 1001, 10111 will be rejected.

Ans. (a)

11. The language $[a^m b^n c^{m+n} \mid m, n \geq 1)$ is

(a) regular
(b) context-free but not regular
(c) context-sensitive but not context-free
(d) type 0 but not context-sensitive

(GATE 2004: 2 Marks)

Solution: The given language can be accepted by PDA, but cannot be accepted by finite automata. So, it is only context-free not regular. Finite automata cannot accept context free language, it only accepts regular language.

Ans. (b)

12. Consider the following grammar G:

$$S \to bS \mid aA \mid b$$
$$A \to bA \mid aB$$
$$B \to bB \mid aS \mid a$$

Let $N_a(w)$ and $N_b(w)$ denote the number of a's and b's in a string w, respectively. The language $L(G) \subseteq \{a,b\}^*$ generated by G is

(a) $\{w|N_a(w) > 3N_b(w)\}$
(b) $\{w|N_b(w) > 3\mathrm{N}_b(w)\}$
(c) $\{w|N_a(w) = 3k,\ k \in \{0, 1, 2, ...\}\}$
(d) $\{w|N_b(w) = 3k,\ k \in \{0, 1, 2, ...\}\}$

(GATE 2004: 2 Marks)

Solution: Strings formed from given grammar are: b, bb, $babaab$, $abbb$, and so on. This means the number of a's are multiples of 3.

According to given grammar, string accepted is $baaa$. Options (a), (b) and (d) are false. Only option (c) is correct.

Ans. (c)

13. L_1 is a recursively enumerable language over Σ. An algorithm A effectively enumerates its words as w_1, w_2, w_3, ... Define another language L_2 over $\Sigma \cup \{\#\}$ as $\{w_i \# w_j: w_j \in L_1,\ i < j\}$. Here # is a new symbol. Consider the following assertions:

S_1: L_1 is recursive implies L_2 is recursive
S_2: L_2 is recursive implies L_1 is recursive

Which of the following statements is true?

(a) Both S_1 and S_2 are true.
(b) S_1 is true but S_2 is not necessarily true.
(c) S_2 is true but S_1 is not necessarily true.
(d) Neither is necessarily true.

(GATE 2004: 2 Marks)

Solution: For L_1, the membership algorithm can be constructed as follows. For an arbitrary $w_i \# w_j$,

(i) If w_i, $w_j \in L_1$ and $i < j$ then $w_i \# w_j$
(ii) If $w_i \notin L_1$ or $w_j \notin L_2$ or $i \geq j$, then $w_i \# w_j \notin L_2$.

This implies that L_2 is also recursive, so S_1 is true. A similar membership algorithm is not possible for L_2, so S_2 is not necessarily true.

Ans. (b)

14. Consider three decision problems P_1, P_2 and P_3. It is known that P_1 is decidable and P_2 is undecidable. Which one of the following is TRUE?

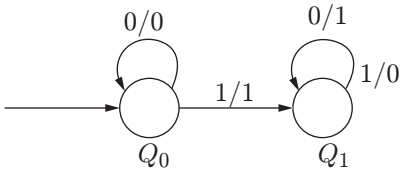
(a) P_3 is decidable if P_1 is reducible to P_3.
(b) P_3 is undecidable if P_3 is reducible to P_2.
(c) P_3 is undecidable if P_2 is reducible to P_3.
(d) P_3 is decidable if P_3 is reducible to P_2's complement.

(GATE 2005, 2 Marks)

Solution: Any problem P is undecidable if any known undecidable problem can be reduced to P. So, option (c) is correct.

Ans. (c)

15. The following diagram represents a finite state machine which takes as input a binary number from the least significant bit.



Which one of the following is TRUE?

(a) It computes 1's complement of the input number
(b) It computes 2's complement of the input number
(c) It increments the input number
(d) It decrements the input number

(GATE 2005: 2 Marks)

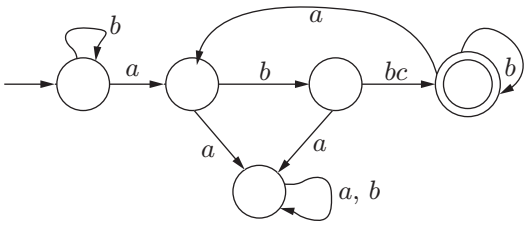
Solution: Output function is:

PS	0	1
0	0	1
1	1	0

The output computes 2's complement of input number.

Ans. (b)

16. Consider the machine M:



The language recognized by M is
(a) $\{w \in \{a, b\}^*$ every a in w is followed by exactly two b's$\}$.
(b) $\{w \in \{a, b)^*$ every a in w is followed by at least two b's$\}$.
(c) $\{w \in (a, b\}^*$ w contains the substring "abb"$\}$.
(d) $\{w \in \{a, b\}^*$ w does not contain "aa" as a substring$\}$.

(GATE 2005: 2 Marks)

Solution: DFA contains every "a" followed by at least two b's due to self-loops.

Ans. (b)

17. Let N_f and N_p denote the classes of languages accepted by non-deterministic finite automata and non-deterministic push-down automata, respectively. Let D_f and D_p denote the classes of languages accepted by deterministic finite automata and deterministic push-down automata, respectively.

Which one of the following is TRUE?

(a) $D_f \subset N_f$ and $D_p \subset N_p$
(b) $D_f \subset N_f$ and $D_p = N_p$
(c) $D_f = N_f$ and $D_p = N_p$
(d) $D_f = N_f$ and $D_p \subset N_p$

(GATE 2005: 2 Marks)

Solution: Power of NFA and DFA is same. But non-deterministic PDA is more powerful than deterministic PDA.

Ans. (d)

18. Consider the languages

$L_1 = \{a^n b^n c^m \mid n, m > 0\}$ and $L_2 = \{a^n b^m c^m \mid n, m > 0\}$

Which one of the following statements is FALSE?

(a) $L_1 \cap L_2$ is a context-free language.
(b) $L_1 \cup L_2$ is a context-free language.
(c) L_1 and L_2 are context-free languages.
(d) $L_1 \cap L_2$ is a context-sensitive language.

(GATE 2005: 2 Marks)

Solution: L_1 and L_2 both are context-free languages. Their intersection may or may not be context-free. So, option (a) is false.

Ans. (a)

19. Let L_1 be a recursive language, and let L_2 be a recursively enumerable but not a recursive language. Which one of the following is TRUE?

(a) $\overline{L_1}$ is recursive and $\overline{L_2}$ is recursively enumerable.
(b) $\overline{L_1}$ is recursive and $\overline{L_2}$ is not recursively enumerable.
(c) $\overline{L_1}$ and $\overline{L_2}$ are recursively enumerable.
(d) $\overline{L_1}$ is recursively enumerable and $\overline{L_2}$ is recursive.

(GATE 2005: 2 Marks)

Solution: Recursive languages are closed under complementation but recursive enumerable languages are not.

Ans. (b)

20. Consider the languages

$$L_1 = \{ww^{\mathrm{R}} \mid w \in \{0, 1\}^*\}$$
$$L_2 = \{w \# w^{\mathrm{R}} \mid w \in \{0, 1\}^*\},$$

where # is a special symbol

$$L_3 = \{ww \mid w \in \{0, 1\}^*\}$$

Which one of the following is TRUE?

(a) L_1 is a deterministic CFL.
(b) L_2 is a deterministic CFL.
(c) L_3 is a CFL, but not a deterministic CFL.
(d) L_3 is a deterministic CFL.

(GATE 2005: 2 Marks)

Solution: L_2 can be accepted by DPDA, so it is a deterministic context free language. L_1 is non-deterministic context free and L_3 in not context free language.

Ans. (b)

21. Let $L_1 = \{0^{n+m}1^n0^m | n, m \geq 0\}$,
$L_2 = \{0^{n+m}1^{n+m}0^m | n, m \geq 0\}$ and
$L_3 = \{0^{n+m}1^{n+m}0^{n+m} | n, m \geq 0\}$

Which of these languages are NOT context-free?

(a) L_1 only (b) L_3 only
(c) L_1 and L_2 (d) L_2 and L_3

(GATE 2006: 1 Mark)

Solution: Only L_1 is accepted by pushdown automata as only L_1 can be written as a context-free language. L_2 is not context free, while L_3 is context sensitive.

Ans. (d)

22. If s is a string over $(0 + 1)^*$, then let $n_0(s)$ denote the number of 0's in s and $n_1(s)$ the number of 1's in s. Which one of the following languages is not regular?

(a) $L = [s \in (0 + 1)^* | n_0(s)$ is a 3-digit prime$\}$
(b) $L = \{s \in (0 + 1)^* |$ for every prefix s' of s, $|n_0(s') - n_1(s')| \leq 2\}$
(c) $L = \{s \in (0 + 1)^* | \, |n_0(s) - n_1(s) \leq 4\}$
(d) $L = \{s \in (0 + 1)^* | n_0(s) \bmod 7 = n_1(s) \bmod 5 = 0\}$

(GATE 2006: 2 Marks)

Solution: Option (a) is finite, can be accepted by DFA.
Option (b), only prefix comparison is possible by DFA, so it is regular.
Option (c), whole string is compared, which is not possible by DFA because it does not have memory.
Option (d), mod machines can be made using DFA.

Ans. (c)

Linked Answer Questions 23 and 24:

23. Which one of the following grammars generates the language $L = \{a^i\, b^i \mid i \neq j\}$?

(a) $S \to AC|CB$
$C \to aCb|a|b$
$A \to aA|\in$
$B \to Bb|\in$

(b) $S \to aS|Sb|a|b$

(c) $S \to AC|CB$
$C \to aCb|\in$
$A \to aA|\in$
$B \to Bb|\in$

(d) $S \to AC|CB$
$C \to aCb|\in$
$A \to aA|a$
$B \to Bb|b$

(GATE 2006: 2 Marks)

Solution: Both options (a) and (b) can produce equal number of a's and b's, which should not be produced according to grammar. Option (d) cannot produce all the strings. So, option (c) is the correct answer.

Ans. (c)

24. In the correct grammar above, what is the length of the derivation (number of steps starring from S) to generate the string $a^l\, b^m$ with $l \neq m$?

(a) max(l, m) + 2 (b) $l + m + 2$
(c) $l + m + 3$ (d) max(l, m) + 3

(GATE 2006: 2 Marks)

Solution: This can be checked by producing various strings of different length.

$$S \to AC$$
$$S \to aC$$
$$S \to aaCb$$
$$S \to aab$$

Here $l = 2$ and $m = 1$, which satisfies the equation max (l, m) + 2 = 4.

Ans. (a)

25. For $s \in (0 + 1)^*$, let $d(s)$ denote the decimal value of s (e.g. $d(101) = 5$).

Let $L = \{s \in (0 + 1)^* \mid d(s) \bmod 5 = 2$ and $d(s) \bmod 7 \neq 4\}$

Which one of the following statements is true?

(a) L is recursively enumerable, but not recursive.
(b) L is recursive, but not context-free.
(c) L is context-free, but not regular.
(d) L is regular.

(GATE 2006: 2 Marks)

Solution: DFA is possible for L. If for any language, we can make DFA then that language will be regular language. So, L is regular.

Ans. (d)

26. Consider the following statements about the context-free grammar:

$$G = \{S \to SS,\ S \to ab,\ S \to ba,\ S \to \varepsilon\}$$

I. G is ambiguous.
II. G produces all strings with equal number of a's and b's.
III. G can be accepted by a deterministic PDA.

Which combination below expresses all the true statements about G?

(a) I only (b) I and III only
(c) II and III only (d) I, II, and III

(GATE 2006: 2 Marks)

Solution:

I. G is ambiguous. Null can be produced using multiple rules [$S \to \varepsilon$ and $S \to SS$]. Two parse trees will be formed.
II. It cannot produce all the combinations of equal a's and b's such as string $aabb$ is not possible.
III. Language accepted is $(ab + ba)^*$, which is regular. Regular languages are also accepted by DPDA.

Ans. (b)

27. Let L_1 be a regular language, L_2 be a deterministic context-free language and L_3 a recursively enumerable, but not recursive, language. Which one of the following statements is false?

(a) $L_1 \cap L_2$ is a deterministic CFL.
(b) $L_3 \cap L_1$ is recursive.
(c) $L_1 \cup L_2$ is context-free.
(d) $L_1 \cap L_2 \cap L_3$ is recursively enumerable.

(GATE 2006: 2 Marks)

Solution:
Option (a) is true. DCFL is closed under regular intersection.
Option (b) is false. Intersection is recursively enumerable.
Options (c) and (d) are true.

Ans. (b)

28. Consider the regular language $L = (111 + 11111)^*$. The minimum number of states in any DFA accepting this languages is

(a) 3 (b) 5
(c) 8 (d) 9

(GATE 2006: 2 Marks)

Solution: Nine states are required. The length of string will be 8. Total states required are 8 + 1 =9.

Ans. (d)

29. Which of the following problems is undecidable?

(a) Membership problem for CFGs
(b) Ambiguity problem for CFGs
(c) Finiteness problem for FSAs
(d) Equivalence problem for FSAs

(GATE 2007: 1 Mark)

Solution: No algorithm exists to check the ambiguity of grammar.

Ans. (b)

30. Which of the following is **TRUE**?

(a) Every subset of a regular set is regular.
(b) Every finite subset of a non-regular set is regular.
(c) The union of two non-regular sets is not regular.
(d) Infinite union of finite sets is regular.

(GATE 2007: 1 Mark)

Solution: In option (a), $a^n b^n$ is subset of a^* b^*; but it is not regular.
Option (b) is correct.
Option (c) is union of two context-free languages, and thus is regular.
Option (d) is infinite union of finite sets, and thus is not regular.

Ans. (b)

31. A minimum state deterministic finite automaton accepting the language

$L = \{w \mid w \in \{0, 1\}^*$, number of 0's and 1's in w are divisible by 3 and 5, respectively$\}$ has

(a) 15 states (b) 11 states
(c) 10 states (d) 9 states

(GATE 2007: 2 Marks)

Solution: Divisible by 3 and $5 = 3 \times 5 = 15$ states are required.

Ans. (a)

32. The language $L = \{0^i\, 21^i \mid i \geq 0\}$ over the alphabet $\{0, 1, 2\}$ is

(a) not recursive
(b) recursive and is a deterministic CFL
(c) a regular language
(d) not a deterministic CFL but a CFL

(GATE 2007: 2 Marks)

Solution: L is accepted by DPDA. So, it is deterministic CFL.

Ans. (b)

33. Which of the following languages is regular?

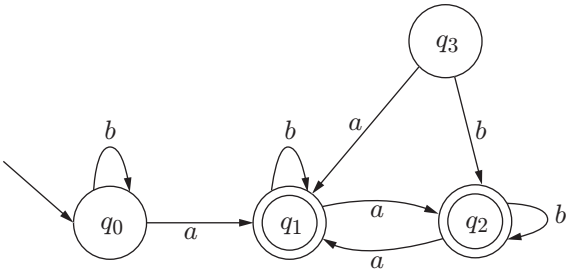
(a) $\{ww^R / w \in \{0, 1\}^+\}$
(b) $\{ww^R x / x, w \in \{0, 1\}^+\}$
(c) $\{wxw^R / x, w \in \{0, 1\}^+\}$
(d) $\{xww^R / x, w \in \{0, 1\}^+\}$

(GATE 2007: 2 Marks)

Solution: Finite automata do not have memory element. So, they cannot remember the symbols. Hence, options (a) and (d) are incorrect.

Ans. (c)

Common Data Questions 34 and 35: Consider the following finite state automaton



34. The language accepted by this automaton is given by the regular expression

(a) $b^*ab^*ab^*ab^*$ (b) $(a + b)^*$
(c) $b^*a\,(a + b)^*$ (d) $b^*ab^*ab^*$

(GATE 2007: 2 Marks)

Solution: One a is compulsory. Minimum string is 'a' that is accepted by the automaton. $b^*a\,(a + b)^*$ is regular and accepted by the automaton.

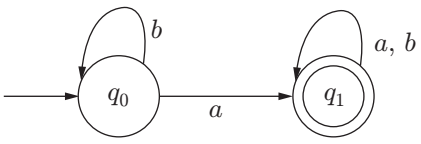
Ans. (c)

35. The minimum state automaton equivalent to the above FSA has the following number of states

(a) 1 (b) 2
(c) 3 (d) 4

(GATE 2007: 2 Marks)

Solution: Minimized DFA is:



Ans. (b)

36. Which of the following is true for the language $\{a^P \mid P \text{ is a prime}\}$?

(a) It is not accepted by a Turing Machine.
(b) It is regular but not context-free.
(c) It is context-free but not regular.
(d) It is neither regular nor context-free, but accepted by a Turing machine.

(GATE 2008: 1 Mark)

Solution: The options are checked by using pumping lemma for regular and context-free languages. It is found that the language is not regular, and hence not context free; but is accepted by the Turing machine.

Ans. (d)

37. Which of the following are decidable?

I. Whether the intersection of two regular languages is infinite.
II. Whether a given context-free language is regular.
III. Whether two pushdown automata accept the same language.
IV. Whether a given grammar is context-free.

(a) I and II (b) I and IV
(c) II and III (d) II and IV

(GATE 2008: 1 Mark)

Solution: Options II and III are undecidable. The intersection of two regular languages is determined by an algorithm, and it can be determined whether the given grammar is context-free or not. So, options I and II are decidable.

Ans. (b)

38. If L and $\bar{L}$ are recursively enumerable, then L is

(a) Regular (b) Context-free
(c) Context-sensitive (d) Recursive

(GATE 2008: 1 Mark)

Solution: A theorem states that when L and $\bar{L}$ both are RE, then L is recursive.

Ans. (d)

39. Which of the following statements is false?

(a) Every NFA can be converted to an equivalent DFA.
(b) Every non-deterministic Turing machine can be converted to an equivalent deterministic Turing machine.
(c) Every regular language is also a context-free language.
(d) Every subset of a recursively enumerable set is recursive.

(GATE 2008: 2 Marks)

Solution: Option (d) is not necessarily true. Every recursive set is recursively enumerable, but the reverse is not necessarily true.

40. Given below are two finite state automata (→ indicates the start state and F indicates the final state)

Y:

	a	b
→1	1	2
2(F)	2	1

Z:

	a	b
→1	2	2
2(f)	1	1

Which of the following represents the product automaton $Z \times Y$?

(a)

	a	b
→P	S	R
Q	R	S
R(F)	Q	P
S	Q	P

(b)

	a	b
→P	Q	S
Q	R	S
R(F)	Q	P
S	Q	P

(c)

	a	b
→P	S	Q
Q	R	S
R(F)	Q	P
S	P	Q

(d)

	a	b
→P	S	Q
Q	S	R
R(F)	Q	P
A	Q	P

(GATE 2008: 2 Marks)

Solution: Option (a) represents the product correctly.

Ans. (a)

41. Which of the following statements are true?

I. Every left-recursive grammar can be converted to a right-recursive grammar and vice versa.
II. All ε productions can be removed from any context-free grammar by suitable transformations.
III. The language generated by a context-free grammar all of whose productions are of the form $X \to w$ or $X \to wY$ (where w is a string of terminals and Y is a non-terminal) is always regular.
IV. The derivation trees of strings generated by a context-free grammar in Chomsky normal form are always binary trees.

(a) I, II, III and IV (b) II, III and IV only
(c) I, III and IV only (d) I, II and IV only

(GATE 2008: 2 Marks)

Solution: Option II: If ε belongs to $L(G)$, then the statement becomes false.

Ans. (c)

42. Match the following.

List I	List II
(E) Checking that identifiers are declared before their use	(P) $L = \{a^n b^m c^n d^m \mid n \geq 1, m \geq 1\}$
(F) Number of formal parameters in the declaration of a function agrees with the number of actual parameters in use of that function	(Q) $X \to XbX \mid XcX \mid dXf \mid g$
(G) Arithmetic expressions with matched pairs of parantheses	(R) $L = \{wcw \mid w \in (a\|b)^*\}$
(H) Palindromes	(S) $X \to bXb \mid cXc \mid \varepsilon$

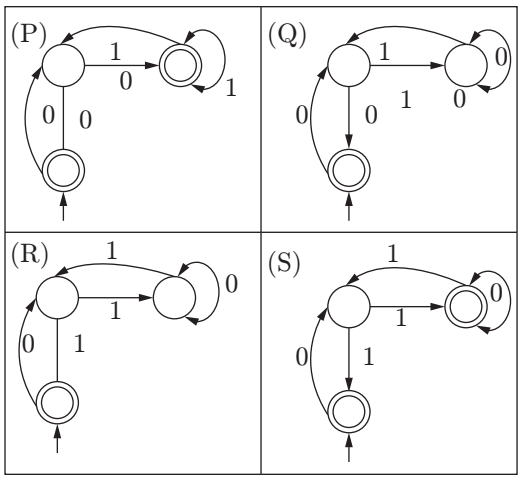
Codes:
(a) E – P, F – R, G – Q, H – S
(b) E – R, F – P, G – S, H – Q
(c) E – R, F – P, G – Q, H – S
(d) E – P, F – R, G – S, H – Q

(GATE 2008: 2 Marks)

Solution:
E – R: In R, first w is checking the declaration of an identifier.
F – P: Actual parameters $a^n b^m$ and formal parameters $c^n d^m$
G –Q: Arithmetic expressions with matched pair of parenthesis
H – S: S is generating palindrome strings.

Ans. (c)

43. Match the following NFAs with the regular expressions they correspond to



1. $\varepsilon + 0(01^*1 + 00)^*01^*$
2. $\varepsilon + 0(10^*1 + 00)^*0$
3. $\varepsilon + 0(10^*1 + 10)^*1$
4. $\varepsilon + 0(10^*1 + 10)^*10^*$

Codes:
(a) P – 2, Q – 1, R – 3, S – 4
(b) P – 1, Q – 3, R – 2, S – 4
(c) P – 1, Q – 2, R – 3, S – 4
(d) P – 3, Q – 2, R – 1, S – 4

(GATE 2008: 2 Marks)

Solution: P $\rightarrow \varepsilon + 0(01^*1 + 00)^*01^*$
Q $\rightarrow \varepsilon + 0(10^*1 + 00)^*0$
R $\rightarrow \varepsilon + 0(10^*1 + 10)^*1$
S $\rightarrow \varepsilon + 0(10^*1 + 10)^*10^*$

Ans. (c)

44. Which of the following are regular sets?

I. $\{a^n b^{2m} | n \geq 0, m \geq 0\}$
II. $\{a^n b^m | n = 2m\}$
III. $\{a^n b^m | n \neq m\}$
IV. $\{xcy | x, y \in (a, b)^*\}$

(a) I and IV only (b) I and III only
(c) I only (d) IV only

(GATE 2008: 2 Marks)

Solution: In options II and III, m and n are dependent. As DFA do not have memory, cannot be accepted by DFA. Options I and IV are regular sets.

Ans. (a)

45. $S \rightarrow aSa|bSb|a|b$

The language generated by the above grammar over the alphabet $\{a, b\}$ is the set of

(a) all palindromes.
(b) all odd length palindromes.
(c) strings that begin and end with the same symbol.
(d) all even length palindromes.

(GATE 2009: 1 Mark)

Solution: Strings generated by this grammar are odd length palindromes, for example, *aaa*, *aba*, *ababa*.

Ans. (b)

46. Which one of the following languages over the alphabet {0, 1} is described by the regular expression (0 + 1)*0(0 + 1)*0(0 + 1)*?

(a) The set of all strings containing the substring 00.
(b) The set of all strings containing at most two 0's.
(c) The set of all strings containing at least two 0's.
(d) The set of all strings that begin and end with either 0 or 1.

(GATE 2009, 1 Mark)

Solution: Given regular expression must have atleast two zeros 0(0 + 1)*0….

Ans. (c)

47. Which one of the following is FALSE?

(a) There is a unique minimal DFA for every regular language.
(b) Every NFA can be converted to an equivalent PDA.
(c) Complement of every context-free language is recursive.
(d) Every non-deterministic PDA can be converted to an equivalent deterministic PDA.

(GATE 2009: 1 Mark)

Solution: Both non-deterministic and deterministic PDA have different powers. Non-deterministic PDA can accept ambiguous grammars also.

Ans. (d)

48. Match all items in Group 1 with correct options from those given in Group 2.

Group 1	**Group 2**
P. Regular expression	**1.** Syntax analysis
Q. Pushdown automata	**2.** Code generation
R. Dataflow analysis	**3.** Lexical analysis
S. Register allocation	**4.** Code optimization

Codes:
(a) P – 4, Q – 1, R – 2, S – 3
(b) P – 3, Q – 1, R – 4, S – 2
(c) P – 3, Q – 4, R – 1; S – 2
(d) P – 2, Q – 1, R – 4, S – 3

(GATE 2009: 1 Mark)

Solution: Regular expressions: Lexical analysis
Pushdown automata: Syntax analysis
Dataflow analysis: Code optimization
Register allocation: Code generation

Ans. (b)

49. Given the following state table of an FSM with two states A and B, one input and one output;

Present State *A*	**Present State *B***	**Input State *C***	**Next State *A***	**Next State *B***	**Output**
0	0	0	0	0	1
0	1	0	1	0	0
1	0	0	0	1	0

(*Continued*)

Present State A	Present State B	Input State C	Next State A	Next State B	Output
1	1	0	1	0	0
0	0	1	0	1	0
0	1	1	0	0	1
1	0	1	0	1	1
1	1	1	0	—	1

If the initial state is $A = 0$, $B = 0$, what is the minimum length of an input string which will take the machine to the state $A = 0$, $B = 1$ with output = 1?

(a) 3 (b) 4 (c) 5 (d) 6

(GATE 2009: 2 Marks)

Solution:

Present State	Output
(0, 0) →	1
(0, 1) →	0
(1, 0) →	0
(0, 1) →	1

With $A = 0$, $B = 1$ and output = 1 will require a string of length 3.

Ans. (a)

50. Let $L = L_1 \cap L_2$, where L_1 and L_2 are languages as defined below:

$$L_1 = \{a^m b^m c a^n b^n \mid m, n^3 \geq 0\}$$
$$L_2 = \{a^i b^i c^k \mid i, j, k \geq 0\}$$

Then L is

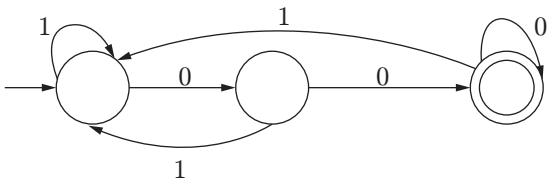
(a) not recursive.
(b) regular.
(c) context-free but not regular.
(d) recursively enumerable but not context-free.

(GATE 2009: 2 Marks)

Solution: Intersection of two regular languages is regular, but L_1 is not regular.

Ans. (c)

51. In the following figure, DFA accepts the set of all strings over {0, 1} that



(a) begin either with 0 or 1
(b) end with 0
(c) end with 00
(d) contain the substring 00

(GATE 2009, 2 Marks)

Solution: The given DFA accepts all the strings ending with 00.

Ans. (c)

52. Let L_1 be a recursive language. Let L_2 and L_3 be languages that are recursively enumerable but not recursive. Which of the following statements is not necessarily true?

(*a*) $L_2 - L_1$ is recursively enumerable.
(*b*) $L_1 - L_3$ is recursively enumerable.
(*c*) $L_2 \cap L_3$ is recursively enumerable.
(*d*) $L_2 \cup L_3$ is recursively enumerable.

(GATE 2010: 1 Mark)

Solution:
Subtraction of recursive language and recursive enumerable is not necessarily recursive enumerable.

Ans. (b)

53. Let $L = \{w \in (0 + 1)^* | w$ has even number of 1's$\}$, i.e., L is the set of all bit strings with even number of 1's. Which one of the regular expressions below represents L?

(a) (0*10*1)* (b) 0*(10*10*)*
(c) 0*(10*1)*0* (d) 0*1(10*1)*10*

(GATE 2010: 2 Marks)

Solution: Option (c) produces odd 1's also, option (a) produces strings ending with 1 and option (d) does not produce zero 1's.

Ans. (b)

54. Consider the languages

$L_1 = \{0^i1^j \mid i \neq j\},\ L_2 = \{0^i1^j \mid i = j\},$

$L_3 = \{0^i1^j \mid i = 2j+1\},\ L_4 = \{0^i1^j \mid i \neq 2j\}$

Which one of the following statements is true?

(a) Only L_2 is context-free
(b) Only L_2 and L_3 are context-free
(c) Only L_1 and L_2 are context-free
(d) All are context-free

(GATE 2010: 2 Marks)

Solution: All are context-free languages as they are recognized by pushdown automata.

Ans. (d)

55. Let w be any string of length n in $\{0, 1\}^*$.
Let L be the set of all substrings of w.
What is the minimum number of states in a non-deterministic finite automaton that accepts L?

(a) $n - 1$ (b) n
(c) $n + 1$ (d) 2^{n-1}

(GATE 2010: 2 Marks)

Solution: $n + 1$ states are required for string of length n.

Ans. (c)

56. Let P be a regular language and Q be a context-free language such that $Q \subseteq P$. (For example, let P be the language represented by the regular expression p^*q^* and Q be $[p^nq^n \mid n \in N]$. Then which of the following is ALWAYS regular?

(a) $P \cap Q$ (b) $P - Q$
(c) $\Sigma^* - p$ (d) $\Sigma^* - Q$

(GATE 2011: 1 Mark)

Solution: Regular languages are closed under complementation.

Ans. (c)

57. The lexical analysis for a modern computer language such as Java needs the power of which one of the following machine models in a necessary and sufficient sense?

(a) Finite state automata
(b) Deterministic pushdown automata
(c) Non-deterministic pushdown automata
(d) Turing machine

(GATE 2011: 1 Mark)

Solution: Tokens are recognized in lexical analysis phase of compiler.

Ans. (c)

58. Which of the following pairs have DIFFERENT expressive power?

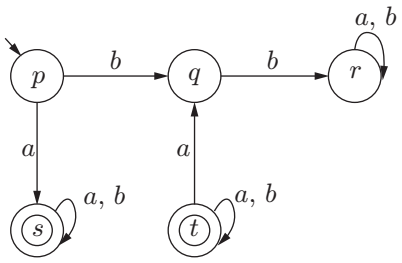
(a) Deterministic finite automata (DFA) and non-deterministic finite automata (NFA)
(b) Deterministic pushdown automata (DPDA) and non-deterministic pushdown automata (NPDA)
(c) Deterministic single-tape Turing machine and Non-deterministic single-tape Turing machine
(d) Single-tape Turing machine and multi-tape Turing machine

(GATE 2011: 1 Mark)

Solution: NDPDA is more powerful than DPDA.

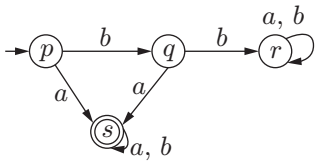
Ans. (b)

59. A deterministic finite automaton (DFA) D with alphabet $\Sigma = \{a, b\}$ is given below.

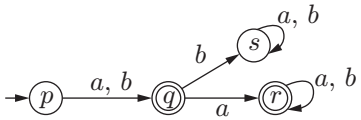


Which of the following finite state machines is a valid minimal DFA which accepts the same language as D?

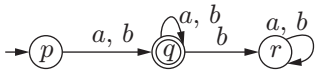
(a)



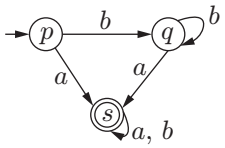
(b)



(c)



(d)



(GATE 2011: 2 Marks)

Solution: Given DFA does not accept "*bba*" which is accepted by option (c) and (d), and string "*b*" accepted by option (b). So, option (a) is answer.

Ans. (a)

60. Definition of a language L with alphabet $\{a\}$ is given as following.

$L = \{a^{nk} \mid k < 0$, and n is a positive integer constant$\}$
What is the minimum number of states needed in a DFA to recognize L?

(a) $(k + 1)$ (b) $(n + 1)$
(c) 2^{k+1} (d) 2^{n+1}

(GATE 2011: 2 Marks)

Solution: Let $n = 4$, so $L = a^{4k}$

Five states are required for this machine. $(n + 1)$ states are required for such a machine.

Ans. (b)

61. Consider the languages L_1, L_2 and L_3 as given below.

$L_1 = \{0^p 1^q | p, q \in N\}$
$L_2 = \{0^p 1^q | p, q \in N$ and $p = q\}$
$L_3 = \{0^p 1^q 0^r | p, q, r \in N, r \in N$ and $p = q = r\}$.

Which of the following statements is NOT TRUE?

(a) Pushdown automata (PDA) can be used to recognize L_1 and L_2.
(b) L_1 is a regular language.
(c) All the three languages are context-free.
(d) Turing machines can be used to recognize all the languages.

(GATE 2011: 2 Marks)

Solution: L_1 is a regular language; L_2 is a context-free language, and L_3 is a context-sensitive language.

Ans. (c)

62. Given the language $L = \{ab, aa, baa\}$, which of the following strings are in L^*?

1. *abaabaaabaa*
2. *aaaabaaaa*
3. *baaaaabaaaab*
4. *baaaaaba*

(a) 1, 2 and 3 (b) 2, 3 and 4
(c) 1, 2 and 4 (d) 1, 3 and 4

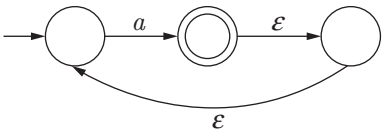
(GATE 2012: 1 Mark)

Solution: *baaaaabaaaab* is not derived from the given language.

Ans. (c)

63. What is the complement of the language accepted by the NFA shown below?

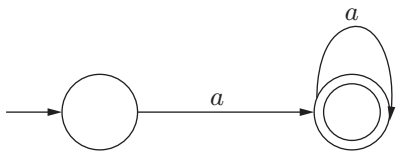
Assume $\Sigma = \{a\}$ and ε is the empty string.



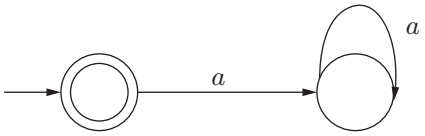
(a) $\varnothing$ (b) $\{\varepsilon\}$
(c) a (d) $\{a, \varepsilon\}$

(GATE 2012: 1 Mark)

Solution: Language accepted by this machine is a^+.
First convert NFA to DFA:



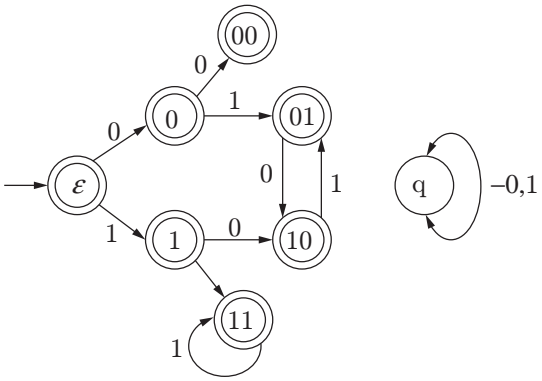
Take compliment of this DFA:



Language accepted is $\{\varepsilon\}$.

Ans. (b)

64. Consider the set of strings on {0, 1} in which *every substring of* 3 *symbols* has at most *two* zeros. For example, 001110 and 011001 are in the language, but 100010 is not. All strings of length less than 3 are also in the language. A partially completed DFA that accepts this language is shown below.



The missing arcs in the DFA are

(a)

	00	01	10	11	q
00	1	0			
01				1	
10	0				
11			0		

(b)

	00	01	10	11	q
00		0			1
01		1			
10				0	
11		0			

(c)

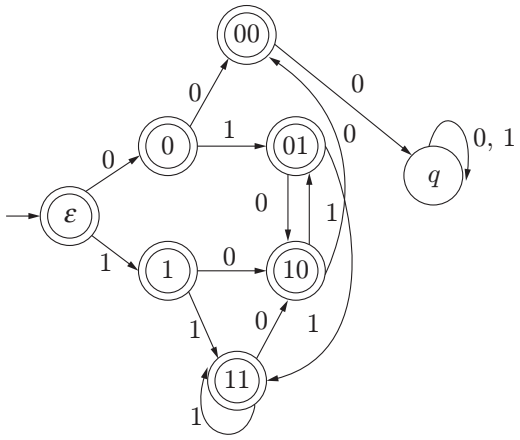
	00	01	10	11	q
00		1			0
01		1			
10			0		
11		0			

(d)

	00	01	10	11	q
00		1			0
01				1	
10	0				
11			0		

(GATE 2012: 2 Marks)

Solution: Complete DFA is given below:



Ans. (d)

65. Consider the following logical inferences.

I_1: If it rains then the cricket match will not be played. The cricket match was played.
Inference: There was no rain.
I_2: If it rains then the cricket match will not be played. It did not rain.
Inference: The cricket match was played.
Which of the following is **TRUE**?

(a) Both I_1 and I_2 are correct inferences
(b) I_1 is correct but I_2 is not a correct inference
(c) I_1 is not correct but I_2 is a correct inference
(d) Both I_1 and I_2 are not correct inferences

(GATE 2012: 2 Marks)

Solution:
R: It rains.
C: Cricket match played.
I_1: $R \rightarrow \sim C$ can be written as $\sim R \cup \sim C$

$$\frac{C}{\sim R}$$

I_2: $R \rightarrow \sim C$ can be written as $\sim R \cup \sim C$

$$\frac{\sim R}{\sim R \cup C}$$

I_1 is correct but I_2 is not a correct inference.

Ans. (b)

66. Which of the following problems are decidable?

1. Does a given program ever produce an output?
2. If L is a context-free language, then, is L also context-free?
3. If L is a regular language, then, is L also regular?
4. If L is a recursive language, then, is L also recursive?

(a) 1, 2, 3, 4 (b) 1, 2
(c) 2, 3, 4 (d) 3, 4

(GATE 2012: 2 Marks)

Solution: Regular and recursive languages are closed under complementation, but CFLs are not.

Ans. (d)

67. Consider the languages $L_1 = \Phi$ and $L_2 = \{a\}$. Which one of the following represents $L_1 L_2^* \cup L_1^*$?

(a) $\{\varepsilon\}$ (b) Φ
(c) a^* (d) $\{\varepsilon, a\}$

(GATE 2013: 1 Mark)

Solution:

$$L_1^* = \varepsilon.\ \textit{So, } L_1 L_2^* \cup L_1^* = \varepsilon$$

Ans. (a)

68. Which of the following statements is/are **FALSE**?

1. For every non-deterministic Turing machine, there exists an equivalent deterministic Turing machine.
2. Turing recognizable languages are closed under union and complementation.
3. Turing decidable languages are closed under intersection and complementation.
4. Turing recognizable languages are closed under union and intersection.

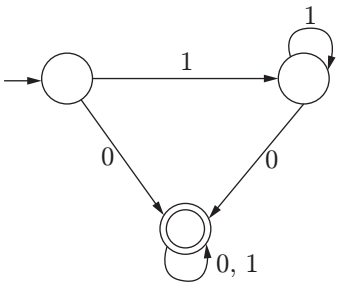
(a) 1 and 4 only (b) 1 and 3 only
(c) 2 only (d) 3 only

(GATE 2013: 1 Mark)

Solution:
Option (a) is true. The power of non-deterministic and deterministic Turing machines is equivalent.
Option (b) is True.
Option (c) is false. They are not closed under complementation.
Option (d) is true.

Ans. (c)

69. Consider the DFA A given below.



Which of the following are **FALSE?**

1. Complement of $L(A)$ is context-free.
2. $L(A) = L((11^*0 + 0)(0 + 1)^*0^*1^*)$
3. For the language accepted by A, A is the minimal DFA.
4. A accepts all strings over $\{0, 1\}$ of length at least 2.

(a) 1 and 3 only (b) 2 and 4 only
(c) 2 and 3 only (d) 3 and 4 only

(GATE 2013, 2 Marks)

Solution: Option (3) is false because minimal DFA can be constructed using two states only. Option (4) is false, DFA accepts string "0", of length 1.

Ans. (d)

70. Which of the following is/are undecidable?

1. G is a CFG. Is $L(G) = \varnothing$?

2. G is a CFG. Is $L(G) = \Sigma^*$?

3. M is a Turing machine. Is $L(M)$ regular?

4. A is a DFA and N is an NFA. Is $L(A) = L(N)$?

(a) 3 only (b) 3 and 4 only
(c) 2 and 3 only (d) 2 and 3 only

(GATE 2013: 2 Marks)

Solution: CFG is empty or not is decidable, and to test whether language accepted by DFA and NFA is same is also decidable.

Ans. (d)

71. Consider the following languages.

$$L_1 = \left\{0^p 1^q 0^r \,|p, q, r \geq 0\right\}$$

$$L_2 = \left\{0^p 1^q 0^r \,|p, q, r \geq 0, p \neq r\right\}$$

Which one of the following statements is **FALSE**?

(GATE 2013: 2 Marks)

(a) L_2 is context-free.
(b) $L_1 \cap L_2$ is context-free.
(c) Complement of L_2 is recursive.
(d) Complement of L_1 is context-free but not regular.

Solution: L_1 is regular and complement of Regular language is also regular.

Ans. (d)

PRACTICE EXERCISES

Set 1

1. What is the meaning of the regular expression Σ^* 0101 Σ^*?

(a) Any string containing "1" as substring
(b) Any string containing "01" as substring
(c) Any string containing "0101" as substring
(d) All strings containing "0101" as substring

2. Which of the following is called a regular operation?

(a) Union, star
(b) Concatenation
(c) Star, concatenation
(d) Concatenation, star, union

3. In Mealy machine, if we enter the string of length 5, then what will be the output string length?

(a) 4 (b) 5
(c) 6 (d) 1

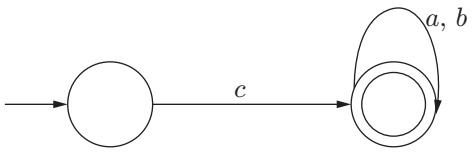
4. The string 1101 does not refer to the set represented by

(a) $110^*(0 + 1)$ (b) $1(0 + 1)^* 101$
(c) $(10)^*(01)^*(00 + 11)$ (d) $(11 + 00) + 01$

5. The grammar $S \to 0S1$, $S \to 0S$, $S \to S1$, $S \to 0$ will generate a

(a) Context-free language
(b) Regular language
(c) Context-sensitive language
(d) Push-Down Automata

6. Which of the string is accepted by a given NFA?



(a) $c.(a \cup b)^*$ (b) $c.a^*b^*$
(c) $c \cdot (ab)^*$ (d) $c \cdot (ab)$

7. Let L denotes the language generated by the grammar $S \rightarrow 0S0|00$. Which of the following is true?

(a) $L = 0^+$
(b) L is regular but not 0^+
(c) L is context-free but not regular
(d) L is context-free

8. Given an arbitrary non-deterministic finite automaton (NDFA) with N states, the maximum number of states in an equivalent minimized DFA is

(a) N^2 (b) 2^N
(c) $2N$ (d) $2^*N!$

9. A language accepted by pushdown automaton is

(a) Regular (b) Context-free
(c) Context-sensitive (d) Recursive

10. Which one of the following is a regular language?

(a) $\{ww \mid n \leq m,\ w \in \{0, 1\}^*\}$
(b) $\{ww^R \mid w \in \{0^*, 1^*\}\}$
(c) $\{w^n w^m \mid n \leq m,\ w \in \{0, 1\}\}$
(d) $\{w(w^R \mid w \in \{0, 1\}^*\}$

11. DFA and NFA when compared on computational power are

(a) DFA $\subseteq$ NFA
(b) NFA $\subseteq$ DFA
(c) DFA = NFA
(d) They cannot be compared

12. Number of states in DFA accepting the strings containing at most $2a$'s and at most $2b$'s over $\{a, b\}$ is

(a) 4 (b) 10
(c) 6 (d) 11

13. Find the minimum number of states in the PDA accepting language

$$L = \{a^n b^m \mid n > m;\ m, n > 0\}$$
$$\Sigma = \{a, b\}, \text{ and } \Gamma = \{z, a\}$$

(a) n^*m (b) 0
(c) 4 (d) 1

14. The context-free grammar which generates the language $L = \{a^n b^m \mid m = n \bmod 3\}$ is

(a) $S \rightarrow AB$
$A \rightarrow aaabbA|\varepsilon$
$B \rightarrow aabb|\varepsilon$

(b) $S \rightarrow AB$
$A \rightarrow AA|\varepsilon$
$B \rightarrow BBBA|\varepsilon$

(c) $S \rightarrow AB$
$A \rightarrow aaaA|\varepsilon$
$B \rightarrow ab|aabb|\varepsilon$

(d) $S \rightarrow BA$
$A \rightarrow BA|\varepsilon$
$B \rightarrow AAAB|\varepsilon$

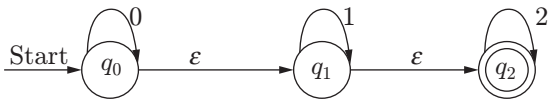
15. Consider the following context-free languages

$L = \{0^m 1^n | m \neq n;\ m, n \geq 0\}$

$L_1 = \{0^m 1^n | m > n \geq 0\}$ and $L_2 = \{0^m 1^n | n > m \geq 0\}$ then

(a) $L = L_1^* L_2$ (b) $L = \varnothing$
(c) $L = L_1 \cup L_2$ (d) $L = L_1 \cap L_2$

16. Consider the following NFA with ε moves



If the given NFA is converted to NFA without ε moves, which of the following denotes the set of final states?

(a) $\{q_2\}$ (b) $\{q_0, q_2\}$
(c) $\{q_0, q_1, q_2\}$ (d) $\{q_1, q_2\}$

17. Which of the following strings will not be accepted by the given NFA in the above question?

(a) 001122 (b) 1122
(c) 0121 (d) 22

18. If L and $\bar{L}$ are recursively enumerable then L is

(a) Regular
(b) Recursive enumerable
(c) Context-sensitive
(d) Recursive

19. Let L_1 be a recursive language. Let L_2 and L_3 be languages that are recursively enumerable but not recursive. Which of the following statements is not necessarily true?

(a) $L_2 - L_1$ is recursively enumerable
(b) $L_1 - L_3$ is recursively enumerable
(c) $L_2 \cap L_1$ is recursively enumerable
(d) $L_2 \cup L_1$ is recursively enumerable

20. Let w be any string of length n in $\{0, 1\}^*$. Let L be the set of all substrings of w. What is the minimum number of states in a non-deterministic finite automaton that accepts L?

(a) $n - 1$ (b) n
(c) $n + 1$ (d) 2^{n-1}

21. A minimum state deterministic finite automaton accepting the language $L = \{w \mid w \in \{0, 1\}^*$, number of 0's and 1's in w are divisible by 3 and 4, respectively$\}$ has

(a) 12 states (b) 11 states
(c) 10 states (d) 8 states

22. $A = \{<M>|$Turing machine that accepts only one string$\}$. The language A is

(a) context-free
(b) recursively enumerable
(c) non-recursively enumerable
(d) regular

23. A push-down automata recognizing $a^n b^n$ has minimum of n states where n is

(a) 3 (b) 0
(c) 1 (d) 2

24. Given the following statements:

S_1: Every context-sensitive language L is recursive.
S_2: There exists a recursive language that is not context-sensitive.

Which statement is correct?

(a) Only S_1 is correct.
(b) Only S_2 is correct.
(c) Both S_1 and S_2 are not correct.
(d) Both S_1 and S_2 are correct.

25. The Greibach normal form grammar for the language $L = \{a^n b^{n+1} \mid n \geq 0\}$ is

(a) $S \rightarrow aSB$, $B \rightarrow bSB|\varepsilon$
(b) $S \rightarrow aSB$, $B \rightarrow bSB|b$
(c) $S \rightarrow aSB|b$, $B \rightarrow b$
(d) $S \rightarrow aSb|b$, $B \rightarrow a$

26. Recursive enumerable languages are not closed under

(a) Union (b) Intersection
(c) Reversal (d) Complement

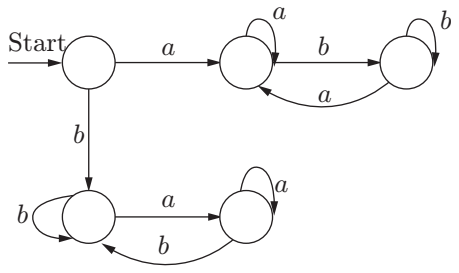
27. Consider the following statements:

I. Recursive languages are closed under complementation.
II. Recursively enumerable languages are closed under complementation.
III. Recursively enumerable languages are closed under union.

Which of the above statements are true?

(a) II only (b) I and III
(c) I and II (d) I, II and III

28. Consider the transition diagram of a DFA as given below



Which should be the final state(s) of the DFA if it should accept strings starting with "a" and ending with "b"?

(a) $\{q_0\}$ (b) $\{q_1\}$
(c) $\{q_0, q_1\}$ (d) $\{q_3\}$

29. Which of the following strings will be accepted in the above case if q_0 and q_1 are accepting states?

1. *ababab*
2. *babaaa*
3. *aaaba*

(a) 1, 2 (b) 2, 3
(c) 1, 3 (d) None of these

30. Which of the following represents the set of accepting states if the language to be accepted contains string having the same starting and ending symbols?

(a) $\{q_0\}$ (b) $\{q_0, q_3\}$
(c) $\{q_3\}$ (d) $\{q_3, q_1\}$

31. If L and $\sim L$ are recursive enumerable then L is

(a) regular and context-free
(b) only context-free
(c) regular and context-sensitive
(d) recursive

32. The following grammar

$G = (N, T, P, S)$
$N = \{S, A, B, C, D, E\}$
$T = (a, b, c\}$
P: $S \rightarrow ABCD$, $BCD \rightarrow DE$, $D \rightarrow aD$, $D \rightarrow a$, $E \rightarrow bE$, $E \rightarrow c$ is

(a) Type 1
(b) Type 1 but not Type 3
(c) Type 2 but not Type 3
(d) Type 0 but not Type 1

33. Context-free languages are closed under

(a) Concatenation and union
(b) Kleene closure and union
(c) Concatenation and union
(d) Concatenation, union, Kleene closure

34. The number of DFAs with the set of states $\{1,2, \ldots n\}$ $(n \geq 1)$ over the alphabet $\{0, 1\}$, with 1 as the initial state is

(a) 2^{n*2} (b) n^2
(c) $n^2 2^{n/2}$ (d) $n^{2n} 2^n$

35. The problem whether the given CFG is finite or infinite is

(a) Undecidable
(b) NP hard and decidable

(c) NP hard
(d) NP complete

36. Let $L = \{w : w$ has $3k + 1$ b's $\forall\ k \geq 0\}$ and a minimized finite automata D accepting L is constructed. The number of states in D will be

(a) 2 (b) 3
(c) 5 (d) 0

37. Which of the following problems are decidable?

1. Does a given program ever produce an output?
2. If L is a context-free language, then is L' (complement of L) also context-free?
3. If L is a regular language, then is L' also regular?
4. If L is a recursive language, then is L' also recursive?

(a) 1, 2, 3, 4 (b) 1, 2,
(c) 2, 3, 4 (d) 3, 4

38. Number of states of the FSM required simulating behaviour of a computer with a memory capable of storing "m" words, each of length "n"

(a) $m \times 2^n$ (b) 2^{mn}
(c) 2^{m+n} (d) All of these

39. Which of the following statement is correct?

(a) $A = \{$If $a^n b^n |\ n = 0, 1, 2, 3, \ldots\}$ is a regular language.
(b) Set B of all strings of equal number of a's and b's denies a regular language.
(c) $L(A^*B^*) \cap B$ gives the set A.
(d) None of these

40. P, Q, R are three languages; if P and R are regular and if $PQ = R$, then

(a) Q has to be regular.
(b) Q cannot be regular.
(c) Q need not be regular.
(d) Q cannot be a CFL.

41. Which of the following definitions below generate the same language as L, where

$$L = \{x^n y^n \text{ such that } n \geq 1\}?$$

1. $E \rightarrow xEy|xy$
2. $xy|(x^+xyy^+)$
3. x^+y^+

(a) 1 only (b) 1 and 2
(c) 2 and 3 (d) 2 only

42. Which of the following statement is correct?

(a) All languages cannot be generated by CFG.
(b) Any regular language has an equivalent CFG.
(c) Some non-regular languages cannot be generated by CFG.
(d) Both (b) and (c).

43. What can be said about a regular language L over $\{a\}$ whose minimal finite state automation has two states?

(a) L must be $\{a^n |\ n$ is odd$\}$
(b) L must be $\{a^n |\ n$ is even$\}$
(c) L must be $\{a^n |\ n > 0\}$
(d) Either L must be $\{a^n |\ n$ is odd$\}$ or L must be $\{a^n |\ n$ is even$\}$

44. Let L be a language recognizable by a finite automation. The language REVERSE $(L) = \{w$ such that w is the reverse of v where $v \in L\}$ is a

(a) Regular language
(b) Context-free language
(c) Context-sensitive language
(d) Recursively enumerable language

45. Consider the following grammar

$S \rightarrow Ax|By$
$A \rightarrow By|Cw$
$B \rightarrow x|Bw$
$C \rightarrow v$

Which of the regular expression describes the same set of strings as the grammar?

(a) $xw^*y + xw^*yx + ywx$
(b) $xwy + xw^*xy + ywx$
(c) $xw^*y + xwxyx + ywx$
(d) $xwxy + xww^*y + ywx$

Set 2

1. Let R_1 and R_2 be regular sets defined over the alphabet, then

(a) $R1 \cap R_2$ is not regular
(b) $R_1 \cup R_2$ is not regular
(c) $\sum^* - R_1$ is regular
(d) ${R_1}^*$ is not regular

2. Which of the following definitions generates the same language as L?

Where $L = \{x^n y^n |\ n \geq 1\}$

(a) $E \rightarrow xEy|xy$ (b) $xy|x^+xyy^+$
(c) $|x^+y^+$ (d) None of these

3. Let $L \subseteq \sum^*$ where $\sum = \{a, b\}$, which of the following is true?

(a) $L = \{x|\ x$ has an equal number of a's and b's$\}$ is regular.

(b) $L = \{a^n\ b^n |\ n \geq 1\}$ is regular.
(c) $L = \{x |\ x$ has more a's than b's$\}$ is regular.
(d) $L = \{a^m\ b^n |\ m \geq n \geq 1\}$ is regular.

4. Which one of the following regular expression over $\{0, 1\}$ denotes the set of all strings not containing 100 as a substring?

(a) 0*(1 + 0)* (b) 0*101*0
(c) 0*1*01* (d) 0*(10 + 1)*

5. If the regular set A is represented by $A = (01 + 1)^*$ and the regular set B is represented by $B = ((01)^*1^*)^*$, which of the following is true?

(a) $A \subset B$
(b) $B \subset A$
(c) A and B are incomparable
(d) $A = B$

6. Let S and T be language over $\sum = \{a, b\}$ represented by the regular expressions $(a + b^*)^*$ and $(a + b)^*$, respectively, which of the following is true?

(a) $S \subset T$ (b) $T \subset S$
(c) $S = T$ (d) $S \cap T = \varnothing$

7. Consider the following statements:

S_1: $\{0^{2n} |\ n \geq 1\}$ is a regular language.
S_2: $\{0^m 1^n 0^{m+n} |\ m \geq 1$ and $n \geq 1\}$ is a regular language.

Which of the following is true about S_1 and S_2?

(a) Only S_1 is correct.
(b) Only S_2 is correct.
(c) Both S_1 and S_2 are correct.
(d) None of S_1 and S_2 is correct.

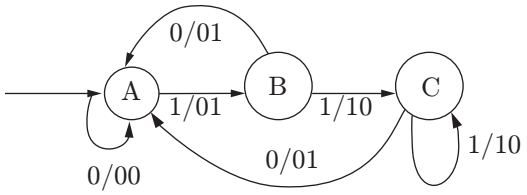
8. Give an arbitrary non-deterministic finite automation (NFA) with N states, the maximum number of states in an equivalent minimized DFA is

(a) N^2 (b) 2^N
(c) $2N$ (d) $N!$

9. Consider a DFA over $\sum = \{a, b\}$ accepting all strings which have number of a's divisible by 6 and number of b's divisible by 8. What is the minimum number of states that the DFA will have

(a) 8 (b) 14
(c) 15 (d) 48

10. The finite state machine described by the following state diagram with A as starting state, where an arc label is x/y and x stands for 1-bit input and y stands for 2-bit output



(a) Outputs the sum of the present and the previous bits of the input
(b) Outputs 01 whenever the input sequence contains 11
(c) Outputs 00 whenever the input sequence contains 10
(d) None of the above

11. The smallest DFA which accepts the language $L = \{x |$ length of x is divisible by $3\}$ has

(a) 2 states (b) 3 states
(c) 4 states (d) 5 states

12. A context-free grammar is ambiguous if

(a) The grammar contains useless on-terminals
(b) It produces more than one parse tree for some sentence
(c) Some production has two non-terminals side by side on the right-hand side
(d) None the above

13. Fortran is a

(a) regular language
(b) context-free language
(c) context-sensitive language
(d) none of these

14. Context-free language and regular languages are both closed under the operation(s) of

(a) union and intersection
(b) intersection and complement
(c) union and concatenation
(d) complementation and concatenation

15. Deterministic context-free languages are

(a) closed under union
(b) closed under complementation
(c) closed under intersection
(d) closed under concatenation

16. Consider the grammar with the following production

$S \rightarrow a\alpha b | b\alpha | ab$
$S \rightarrow \alpha S | b$
$S \rightarrow \alpha bb | ab$
$S\alpha \rightarrow bdb | b$

The above grammar is

(a) context-free grammar
(b) regular grammar
(c) context-sensitive grammar
(d) LR(k)

17. If L_1 and L_2 are context-free languages and R a regular set, one of the languages below is not necessarily a context-free language. Which one?

(a) L_1L_2 (b) $L_1 \cap L_2$
(c) $L_1 \cap R$ (d) $L_1 \cup L_2$

18. Which of the following statement is false?

(a) Every finite subset of a non-regular set is regular.
(b) Every subset of a regular set is regular.
(c) Every finite subset of a regular set is regular.
(d) The intersection of two regular sets is regular.

19. If two finite state machines are equivalent, they should have the same number of

(a) states (b) edges
(c) states and edges (d) none of these

20. Context-free languages are closed under

(a) union, intersection
(b) union, Kleene closure
(c) intersection, complement
(d) eomplement, Kleene closure

21. Let L_D be the set of all languages accepted by a PDA by final state and L_E the set of all languages accepted by empty stack. Which of the following is true?

(a) $L_D = L_E$ (b) $L_D \supset L_E$
(c) $L_D \subset L_E$ (d) None of these

22. If L_1 is a context-free language and L_2 is regular, which of the following is false?

(a) $L_1 - L_2$ is not context-free.
(b) $L_1 \cap L_2$ is context-free.
(c) ~L_1 is not context-free.
(d) ~L_2 is regular.

23. The language accepted by pushdown automation in which the stack is limited to 10 items is best described as

(a) context-free
(b) regular
(c) deterministic context-free
(d) recursive

24. Recursive languages are

(a) a proper superset of context-free languages
(b) also called 0 languages
(c) recognizable by Turing machines
(d) all of the above

25. The productions

$E \rightarrow E + E$
$E \rightarrow E - E$
$E \rightarrow E^*E$
$E \rightarrow E/E$
$E \rightarrow id$

(a) generate an inherently ambiguous language
(b) generate an ambiguous language but not inherently so
(c) are unambiguous
(d) can generate all possible fixed length valid computation for carrying out addition, subtraction, multiplication and division, which can be expressed in one expression

26. In which of the cases stated below is the following statement true?

"For every non-deterministic machine M_1 there exists an equivalent deterministic machine M_2 recognizing the same language"

I. M_1 is non-deterministic finite automation
II. M_1 is a non-deterministic PDA
III. M_1 is a deterministic Turing machine
IV. there is no machine M_1 for which the given statement is true

(a) I and II (b) I and III
(c) I, II and III (d) II and IV

27. Which of the following conversion is not possible (algorithmically)?

(a) Regular grammar to context-free grammar
(b) Non-deterministic FSA to deterministic FSA
(c) Non-deterministic PDA to deterministic PDA
(d) Non-deterministic Turing machine to deterministic Turing machine

28. Which one of the following is not decidable?

(a) Give a Turing machine M, a string s and an integer k, M accepts s within k steps
(b) Equivalence of two Turing machines
(c) Languages accepted by a given finite state machine is non-empty
(d) Languages accepted by a context-free grammar is non-empty

29. Which of the following is true?

(a) The complement of a recursive language is recursive.

(b) The complement of a recursively enumerable language is recursively enumerable.
(c) The complement of a recursive language is either recursive or recursively enumerable.
(d) The complement of a context-free language is context-free.

30. The C language is

(a) A context-free language
(b) A context-sensitive language
(c) A regular language
(d) Parsable fully only by a Turing machine

31. Any string of terminals that can be generated by the following CFG is

$S \rightarrow XY$
$X \rightarrow aX|bX|a$
$Y \rightarrow Ya|Yb|a$

(a) Has at least one "b"
(b) Should end in "a"
(c) Has no consecutive a's or b's
(d) Has at least two a's

32. Which of the following problems are undecidable?

(I) Membership problem in context-free languages
(II) Whether a given context-free language is regular
(III) Whether a finite state automation halts on all inputs
(IV) Membership problem for type 0 languages

(a) II and IV (b) I and II
(c) III and IV (d) I and IV

33. For two regular languages

$$L_1 = (a + b)^*a \text{ and } L_2 = b(a + b)^*$$

The intersection of L_1 and L_2 is given by

(a) $(a + b)^*ab$ (b) $ab(a + b)^*$
(c) $a(a + b)^*b$ (d) $b(a + b)^*a$

34. Consider the following problem X.

"Given a Turing machine M over the input alphabet $\sum$, any state q of M and a word $\sum^*$, does the computation of M on w visit the state q"

Which of the following statements about X is correct?

(a) X is decidable.
(b) X is undecidable but partially decidable.
(c) X is undecidable and not even partially decidable.
(d) X is not a decision problem.

ANSWERS TO PRACTICE EXERCISES

Set 1

1. (d)	**9.** (c)	**17.** (c)	**25.** (c)	**33.** (d)	**41.** (a)
2. (d)	**10.** (d)	**18.** (d)	**26.** (c)	**34.** (d)	**42.** (d)
3. (b)	**11.** (c)	**19.** (b)	**27.** (b)	**35.** (a)	**43.** (b)
4. (c)	**12.** (b)	**20.** (c)	**28.** (b)	**36.** (b)	**44.** (a)
5. (b)	**13.** (c)	**21.** (a)	**29.** (c)	**37.** (d)	**45.** (a)
6. (a)	**14.** (c)	**22.** (b)	**30.** (b)	**38.** (b)	
7. (b)	**15.** (c)	**23.** (a)	**31.** (d)	**39.** (c)	
8. (b)	**16.** (c)	**24.** (d)	**32.** (d)	**40.** (c)	

Set 2

1. (c) Regular languages are closed under union, interaction, Kleene closure and complementation. So option (c) is correct and remaining options (a), (b) and (d) are false.

2. (a) $L = \{x^n y^n | n \geq 1\}$

The given language L produces all strings having equal number of x and y, where y follows symbol x. Only expression $\Rightarrow E \rightarrow Ey \,|\, xy$ can produce equal number of x and y, where y follows x.

3. (d) Regular language is accepted by DFA. DFA does not have memory, so it is not able to do comparison between two values, that they are equal or not (means equal number of a's and b's). So only option (d) does not have comparison, which means it can be accepted by DFA.

4. (a)

a. 0*(1 + 0)* can generate string 0000100, which contains substring 100.
b. 0*101*0 generates strings which does not contain 100 as substring.
c. 0*1*01* generates strings which does not contain 100 as substring.
d. 0*(10+1)* generates all strings which does not contain 100 as substring.

5. (d) Strings in language generated by both the regular expression is equivalent.

So, $A = B$

6. (c) Language generated by both the regular expression is equivalent, so $(a + b^*)^* = (a + b)^*$.

7. (a) $L_1 = \{0^{2n} | \ n \geq 1\}$, L_1 is regular language because it produces even number of 0's. $L_2 = \{0^m 1^n 0^{m+n} | \ m \geq 1 \text{ and } n \geq 1\}$, L_2 is context-free language because there are two comparisons. So, S_1 is correct but S_2 is not correct.

8. (b) NFA with N states, an equivalent minimized DFA having maximum 2^N states.

9. (d) A DFA over $\sum = \{a, b\}$ accepting all strings which have number of a's divisible by m and number of b's divisible by n have m^*n states. In our example we have $6 \times 8 = 48$ states.

10. (a) FSM outputs the sum of the present and previous bits of the input. Let us suppose input is 101, then output is 0 + 1 (last 2 bits) = 01. When input is 11 then output is 1 + 1 = 10.

11. (b) The minimal DFA which accepts the language $L = \{x | \text{ length of } x \text{ is divisible by } n\}$ always have n states.

12. (b) A context-free grammar is ambiguous if there is a word which has two different derivation trees.

13. (c) Fortran is a context-sensitive languages.

14. (c) Regular languages are closed under the operation of union, interaction, reversal, concatenation and complementation. Context-free languages are closed under the operation of concatenation and union.

15. (b) Deterministic context-free languages are closed under complementation.

16. (c)

$S \rightarrow a\alpha b | b\alpha | ab$
$S \rightarrow \alpha S | b$
$S \rightarrow \alpha bb | ab$
$S\alpha \rightarrow bdb | b$

The given production ($S\alpha \rightarrow bdb|b$) contains terminal on the left-hand side, which means this grammar cannot be context-free grammar and regular. So, this grammar is context-sensitive.

17. (b) If L_1 and L_2 are context-free language then they are closed under

(i) Concatenation L_1L_2
(ii) Union $L_1 \cup L_2$
(iii) Under regular intersection $L_1 \cap R$

But they are not closed under intersection $L_1 \cap L_2$.

18. (b) For (a) and (c), if a set is finite then it means we can create DFA for that. It does not matter that it belongs to non-regular or regular. If we are able to draw DFA for that set then it means that it is regular. So every finite set of regular or non-regular is regular.

For option (d), regular language is closed under intersection.

Let $\sum = (a, b)$, $\sum^*$ is a regular set.

$L_1 = \{a^n b^n | \ n \geq 0\}$

$L_2 = \{a^n b^n | \ nm \geq 0\}$

Here, $L_1 \subset \sum^*$ and $L_2 \subset \sum^*$

But L_1 is CFL and L_2 is regular. So from the above explanation, every subset of regular set need not be regular.

19. (d) Two finite state machines are said to be equivalent if they can accept the same language.

20. (b) Context-free languages are closed under union and Kleene closure and not closed under intersection and complement.

21. (a) PDA accepts its input by two ways:
(a) Accepting by entering into final state
(b) Accepting by empty stack

The acceptance or recognition power of PDA of both approaches are same because there exists algorithm to convert PDA accepting by empty stack to PDA accepting by final state and vice versa, Therefore $L_D = L_E$.

22. (a) L_1 = Context-free language (CFL)

L_2 = Regular
$L_1 - L_2 = L_1 \cap L_2^C$

$L_2{}^C$ is a regular language because regular is closed under complement and so $L_1 \cap L_2{}^C$ is context-free because languages are closed under regular intersection. CFL is closed under regular intersection, so $L_1 \cap L_2$ is a context-free language. CFL is not closed under complementation, so $\sim L_1$ is not a context-free language. Regular languages are closed under complementation, so $\sim L_1$ is regular.

23. (b) A PDA with a stack limited to containing limited items is equivalent to a DFA, and language accepted by DFA is regular. So, regular is the best word to describe it.

24. (d) Recursive language is a superset of CFL. Recursive language is called type 0 or unrestricted language and accepted by Turing machine.

25. (b) Given grammar will generate ambiguous grammar because for a single string there can be two parse tree according to a given grammar.

26. (b)

(a) Power of NFA and DFA is same because there exists an algorithm to convert every NFA to DFA.
(b) Recognition power of NPDA and DPDA are not same because there is no algorithm to convert NPDA to DPDA.
(c) Power of NTM and DTM is same because there exists an algorithm to convert every NTM to DTM.
(d) Since for finite automaton and Turing machines the above statements are true, so this option is false.

27. (c) Since recognition power of NPDA (non-deterministic PDA) and DPDA (deterministic PDA) are not same because there is no algorithm to convert NPDA to DPDA. Means there exists some languages such as $L = \{ww^R | \ w \in (a, b^*\}$ which are accepted by NPDA but not by DPDA.

28. (b)

(a) Give a Turing machine M, a string s and an integer k, M accepts s within k steps is a decidable problem.
(b) Equivalence of two Turing machines is an undecidable problem.
(c) Emptiness of regular language is a decidable problem.
(d) Emptiness of CFL's is a decidable problem.

29. (a) Recursive languages are closed under complementation so complement of a recursive language is also recursive. As every recursive language is also recursive enumerable. So the complement of recursive languages is also recursive enumerable. So both options (a) and (c) are true but (a) is the strongest answer.

30. (b) The C language is a context-sensitive language.

31. (d) Has at least 2 numbers of a's

32. (a) Only options (II) and (IV) are undecidable because there exists no algorithm to check that a CFL is regular and also for membership problem for RE language.

33. (d) Language $L_1 = \{(a + b)^* a\}$ is a set of strings possible with "a" and "b" and which ends with terminal "a".
Language $L_2 = \{b(a + b)^*\}$ is a set of all strings possible with "a" and "b" and which starts with terminal "b".
So their intersection will give set of strings which starts with "a" and ends with "b", that is, $b(a + b)^* a$.

34. (b) There is algorithm to reduce halting problem of Turing machine to state entry problem. The given problem X is a state-entry problem. As halting problem of Turing machine is undecidable, so problem X also must be undecidable. The language corresponding to state-entry problem is RE but not REC. So X is undecidable, but partially decidable.